

MENTOR: HOÀNG BẢO LONG

ARITHMETIC LOGIC UNIT 8BIT

THỰC HIỆN:

LÊ KHẮC NGHĨA - 23DT3

ĐINH BẢO - 24VDT

LÊ QUỐC ĐẠT - 24VDT

PHÂN CHIA CÔNG VIỆC

1

- SYNTHESIS
- THIẾT KẾ POWERPOINT

LÊ KHẮC NGHĨA

2

- CODE RTL & TESTBENCH
- HIGH LEVEL DESIGN
- LOW LEVEL DESIGN

LÊ QUỐC ĐẠT

3

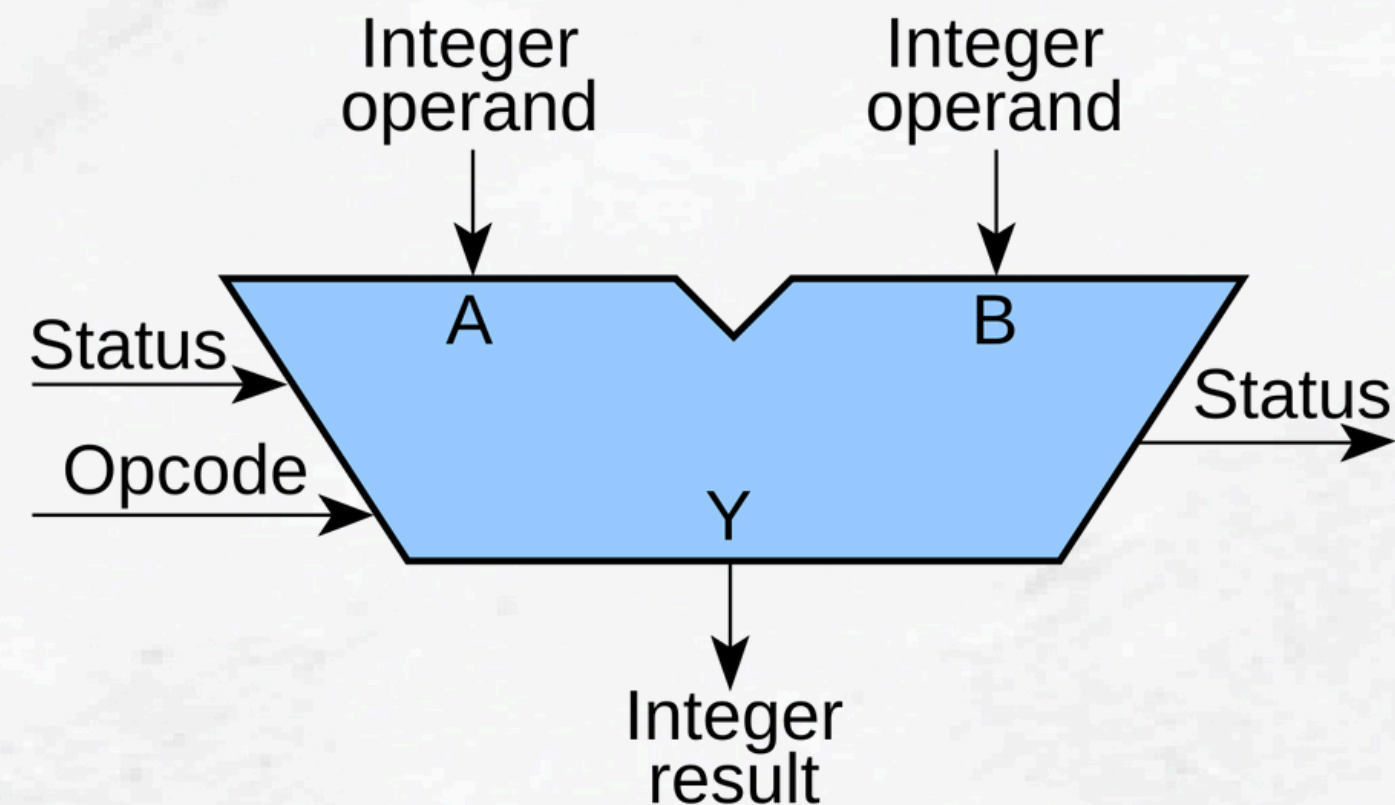
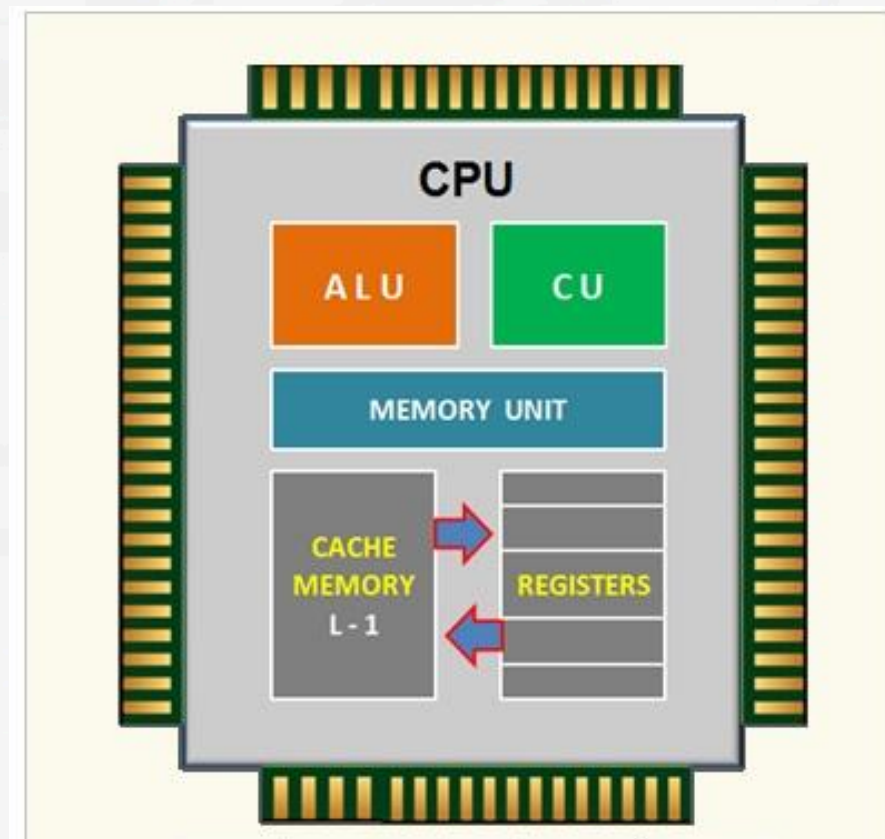
- CODE RTL & TESTBENCH
- HIGH LEVEL DESIGN
- LOW LEVEL DESIGN

ĐINH BẢO

1.INTRODUCTION

An arithmetic-logic unit is the part of a Central Processing Unit that carries out arithmetic and logic operations on the operands in computer instruction words.

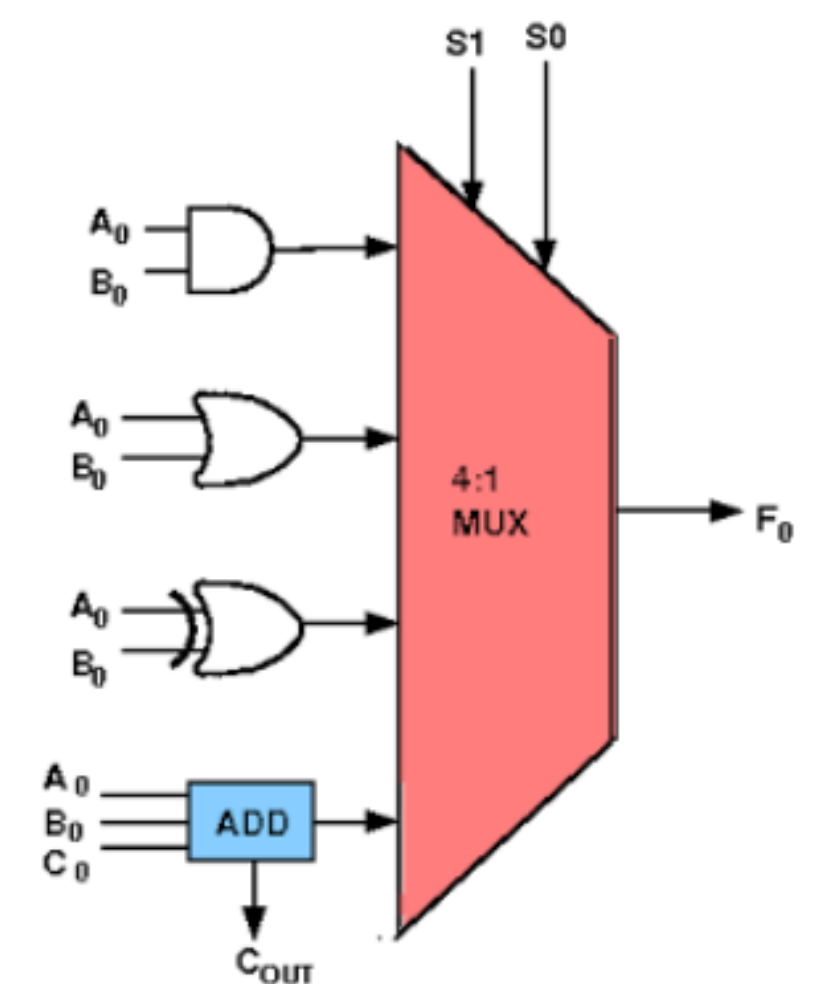
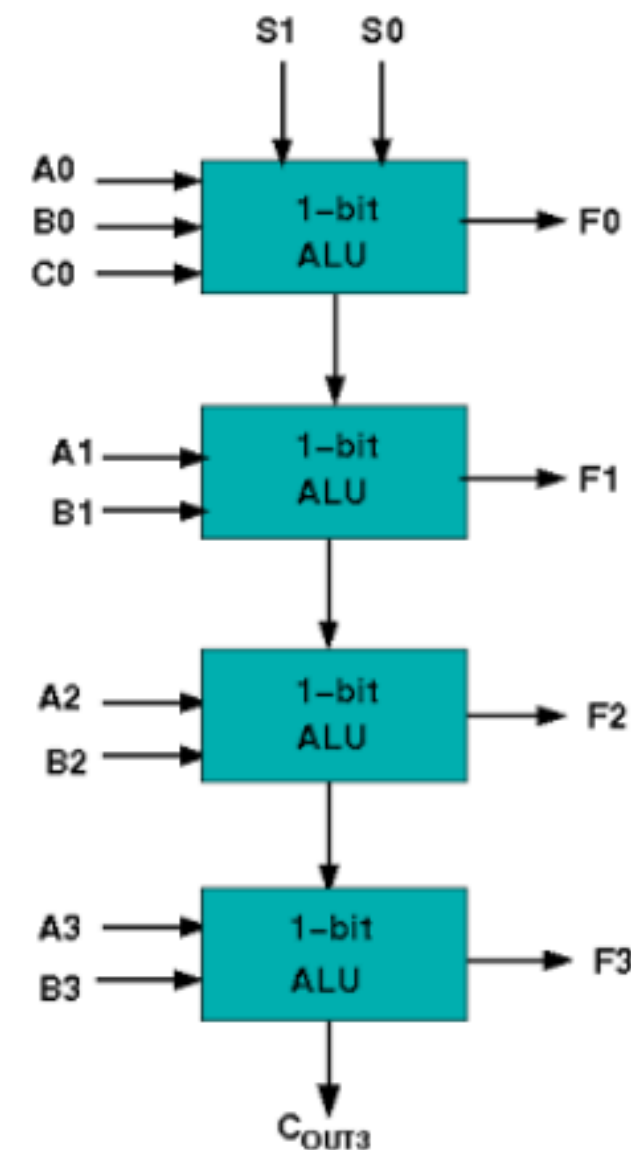
In computer science, ALUs serve as a **combinational digital circuit** that performs arithmetic and bitwise operations on binary numbers. This is a foundational building block of arithmetic logic circuits for numerous types of control units and computing circuits including central processing units (CPUs), FPUs and graphics processing units.



1.INTRODUCTION

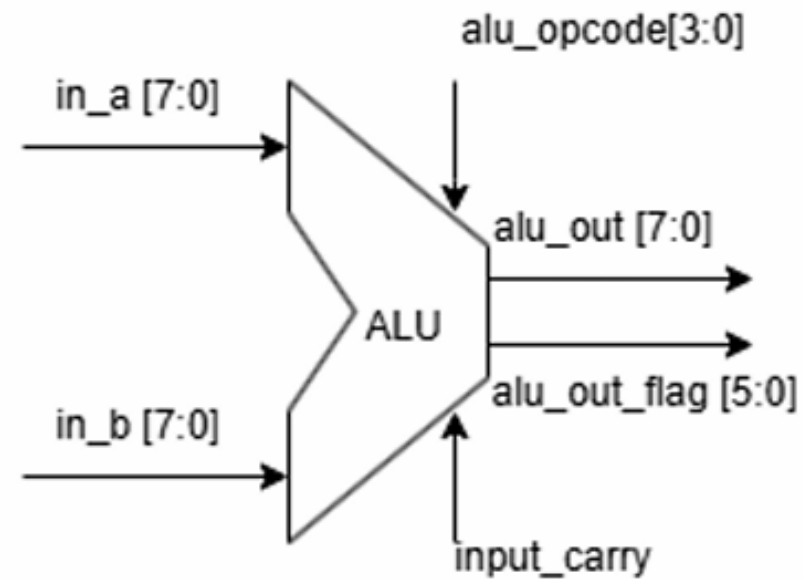
The following are a few examples of bitwise logical operations and basic arithmetic operations supported by ALUs:

- Addition: Adds A and B with carry-in or carry-out sum at Y.
- Subtraction: Subtracts B from A or vice versa with the difference at Y and carry-in or carry-out.
- Increment: Where A or B is increased by one and Y represents the new value.
- Decrement: Where A or B is decreased by one and Y represents the new value.
- AND: The bitwise logic AND of A and B is represented by Y.
- OR: The bitwise logic OR of A and B is represented by Y.
- Exclusive-OR: The bitwise logic XOR of A and B is represented by Y



2. DESIGN SPECIFICTION

2. DESIGN SPECIFICATION



ALU 8 bit block

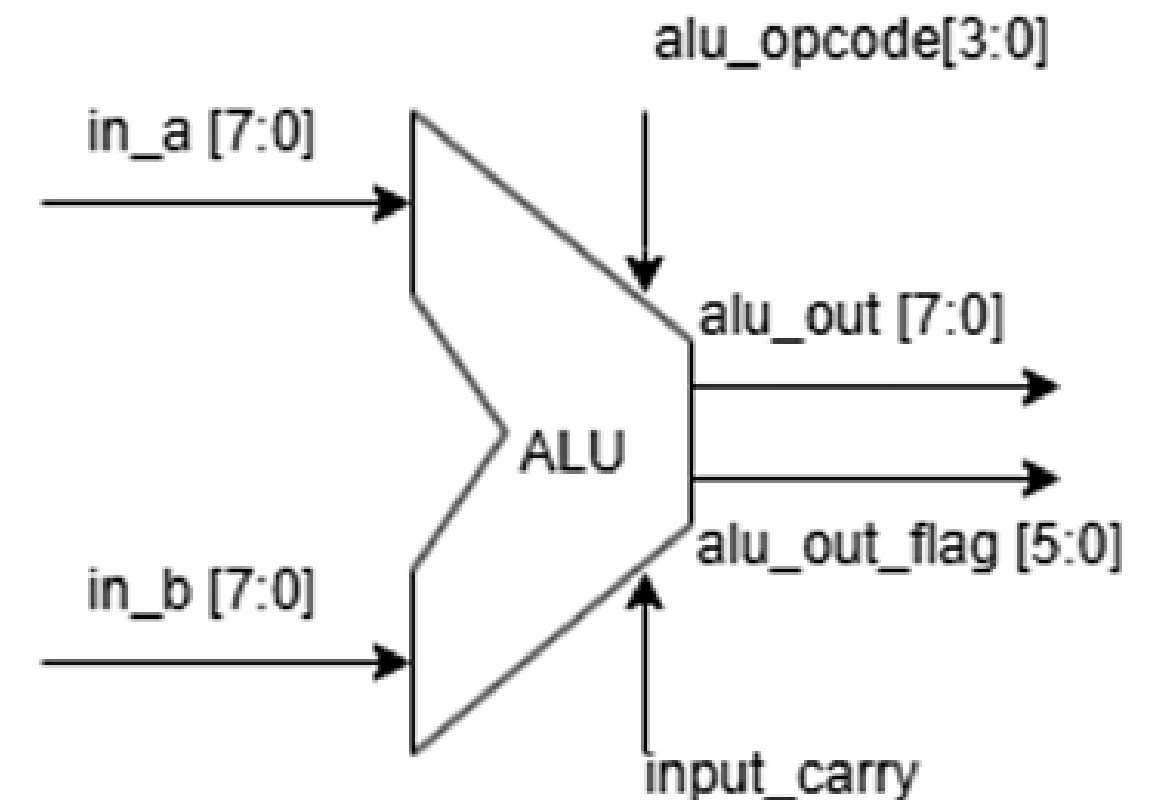
Port	Size	Direction	Description
in_a	8	input	value input a
in_b	8	input	value input b
input_carry	1	input	value input carry
alu_opcode	4	input	code for ALU operation
alu_out	8	output	output of ALU block
alu_out_flag	5	output	flag output of ALU block

2. DESIGN SPECIFICATION

2.1. INTERFACE

The first bit of the code input acts as a mode selection for ALU. Remaining 3 are used for the ALU operation.

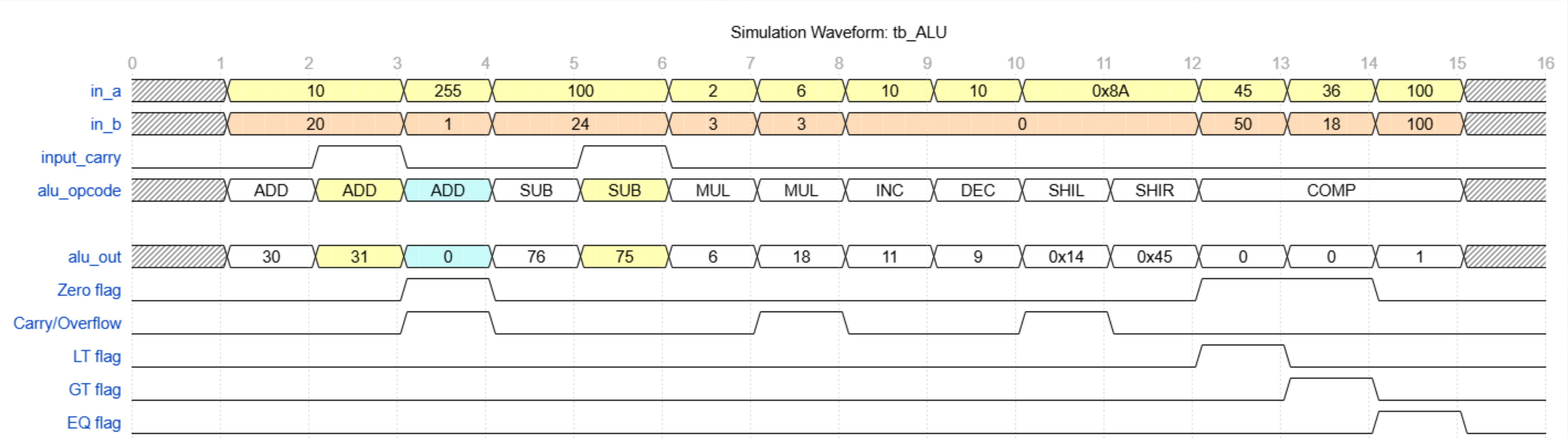
- 0000 - Mode 0, Arithmetic Unit for ADD
- 0001 - Mode 0, Arithmetic Unit for Subtract
- 0010 - Mode 0, Arithmetic Unit for Multiply
- 0011 - Mode 0, Arithmetic Unit for Increase
- 0100 - Mode 0, Arithmetic Unit for Decrease
- 0101 - Mode 0, Arithmetic Unit for Shift left
- 0110 - Mode 0, Arithmetic Unit for Shift right
- 1000 - Mode 1, Logical Unit for Compare
- 1001 - Mode 1, Logical Unit for AND
- 1010 - Mode 1, Logical Unit for OR
- 1011 - Mode 1, Logical XOR
- 1100 - Mode 1, Logical NOT A
- 1101 - Mode 1, Logical NOT B



ALU 8 bit block

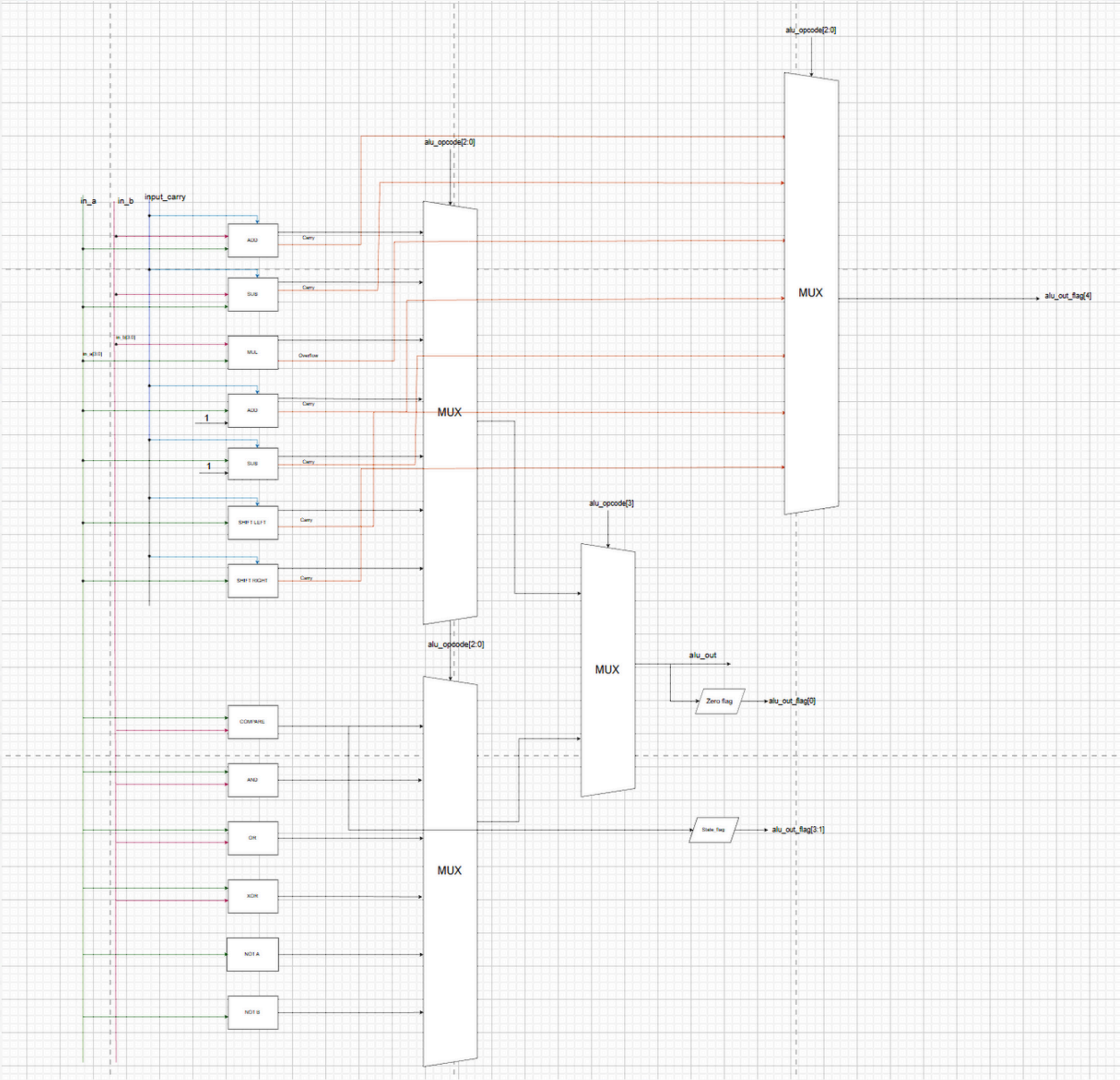
2. DESIGN SPECIFICATION

2.1. Interface



2. DESIGN SPECIFICATION

2.2. High Level Block Design



2. DESIGN SPECIFICATION

2.2. High Level Block Design

2.2.1. Arithmetic Unit Block

Alu operation code for Arithmetic unit: alu_opcode:

0000 - Mode 0, Arithmetic Unit for ADD

0001 - Mode 0, Arithmetic Unit for Subtract

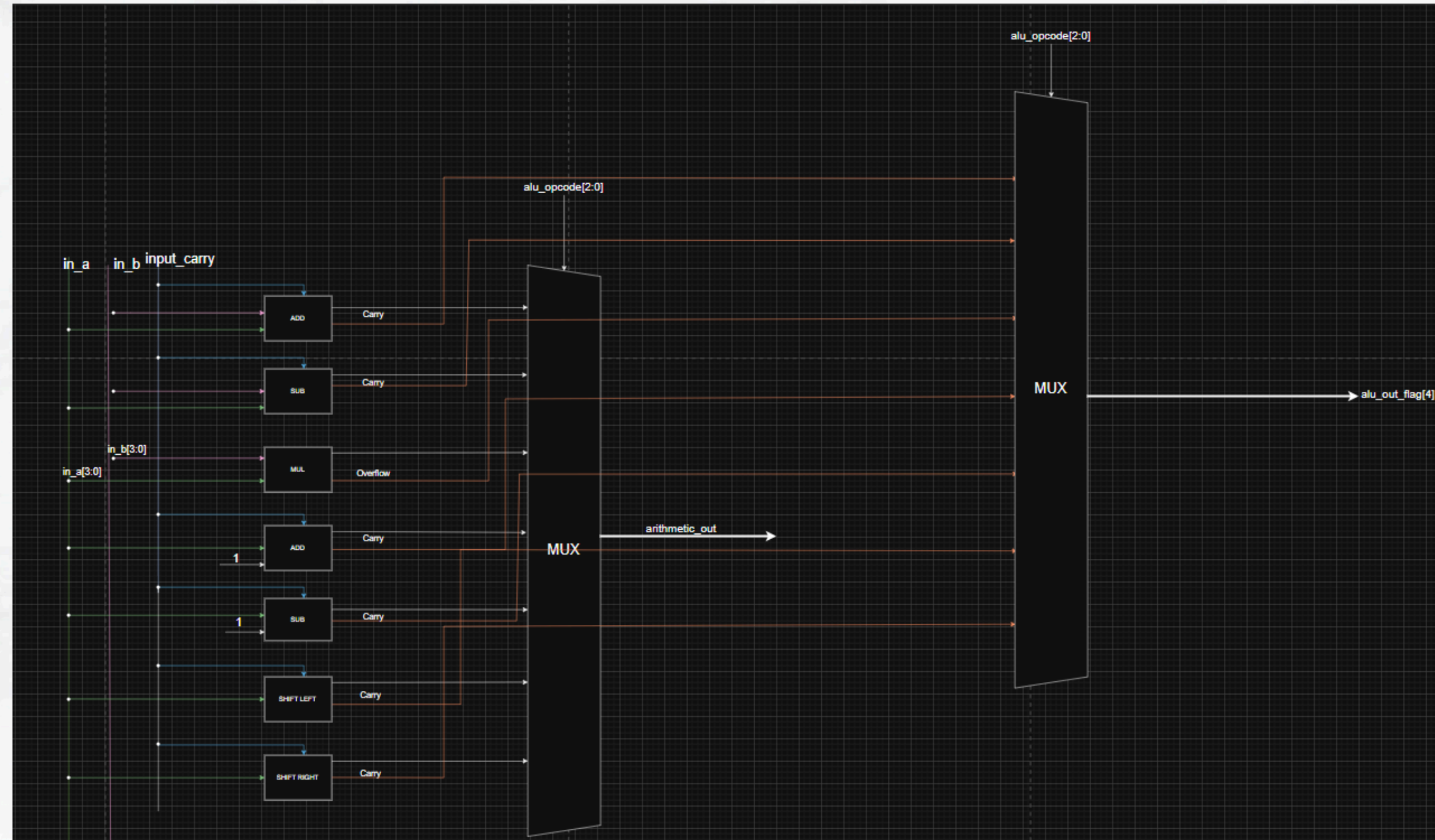
0011 - Mode 0, Arithmetic Unit for Multiply

0100 - Mode 0, Arithmetic Unit for Increase

0101 - Mode 0, Arithmetic Unit for Decrease

0110 - Mode 0, Arithmetic Unit for Shift left

0111 - Mode 0, Arithmetic Unit for Shift right



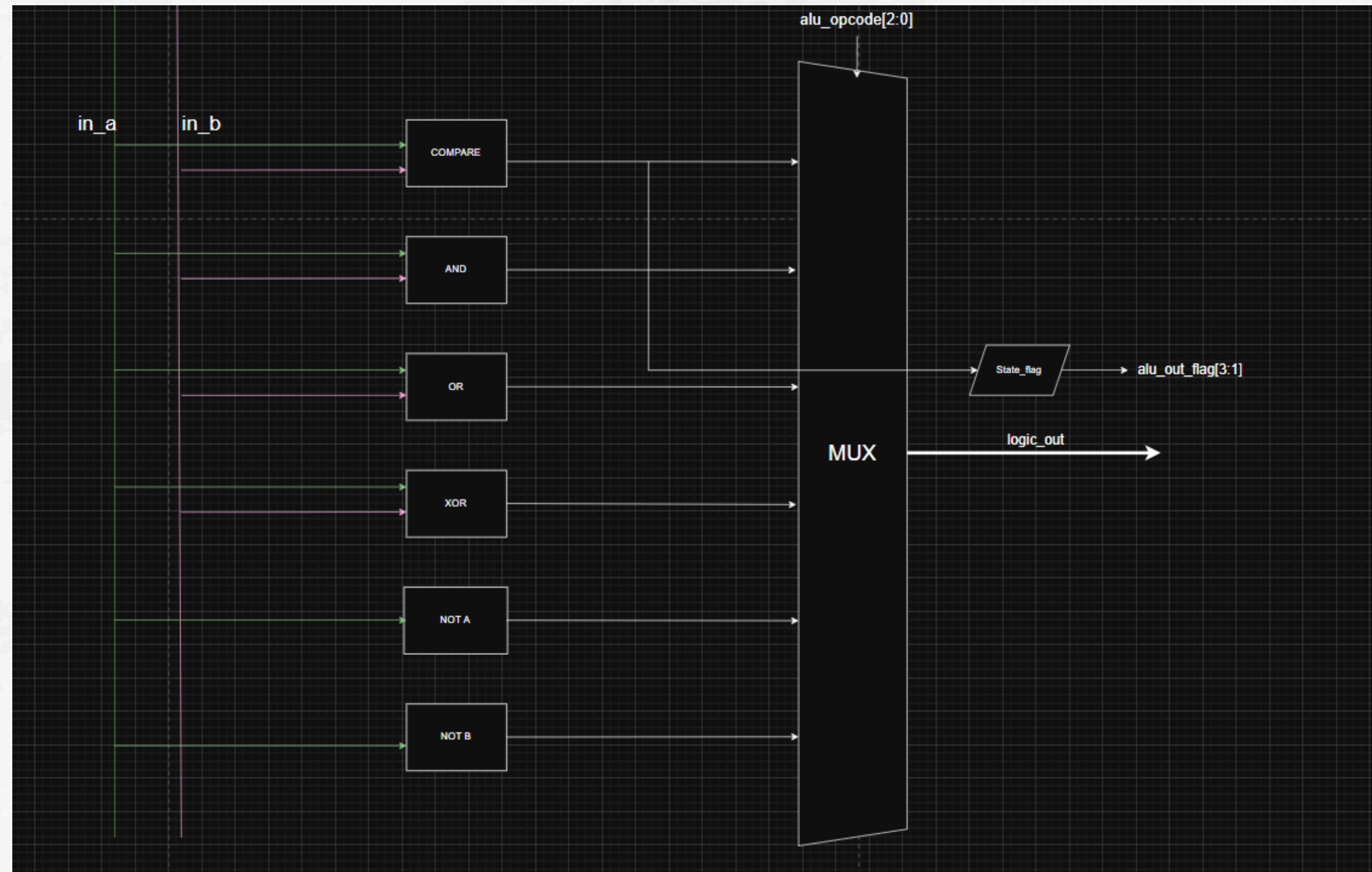
2. DESIGN SPECIFICATION

2.2. High Level Block Design

2.2.2. Logic Unit Block

Alu operation code for Logic unit:
alu_opcode:

- 1000 - Mode 1, Logical Unit for Compare
- 1001 - Mode 1, Logical Unit for AND
- 1010 - Mode 1, Logical Unit for OR
- 1011 - Mode 1, Logical XOR
- 1100 - Mode 1, Logical NOT A
- 1101 - Mode 1, Logical NOT B



2. DESIGN SPECIFICATION

2.3. Low Level Block Design

2.3.1. Arithmetic Unit Block

a. Full-Adder

A	B	Cin	S	Cout
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

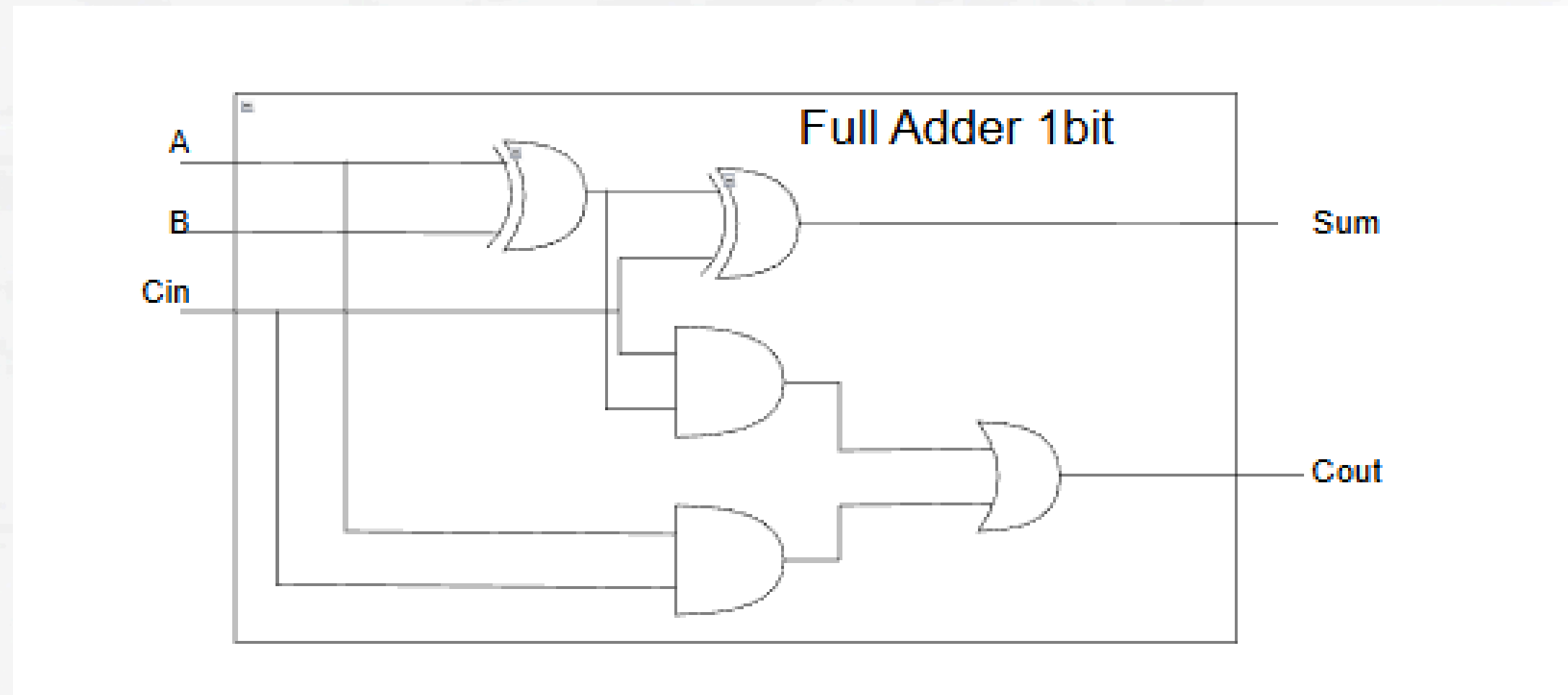
True Table of 1 bit Full Adder

2. DESIGN SPECIFICATION

2.3. Low Level Block Design

2.3.1. Arithmetic Unit Block

a. Full-Adder



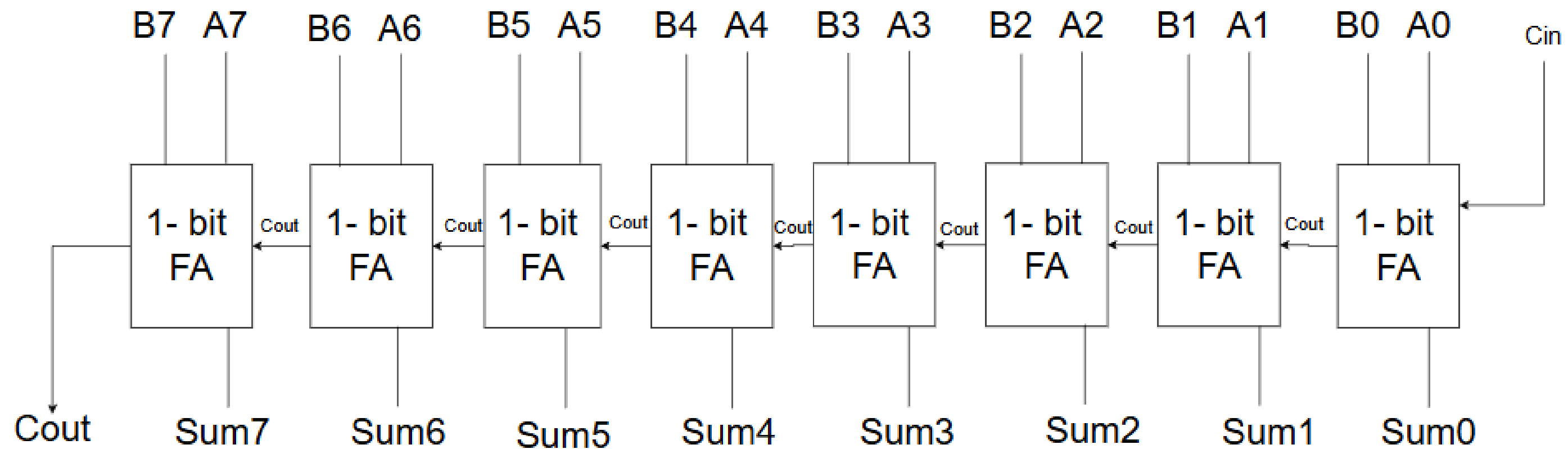
1 bit Full-Adder

2. DESIGN SPECIFICATION

2.3. Low Level Block Design

2.3.1. Arithmetic Unit Block

a. Ripper carry Full-Adder



8 bit Full-Adder

2. DESIGN SPECIFICATION

2.3. Low Level Block Design

2.3.1. Arithmetic Unit Block

b. Full Suctractor

A	B	Bin	D	Bout
0	0	0	0	0
0	0	1	1	1
0	1	0	1	1
0	1	1	0	1
1	0	0	1	0
1	0	1	0	0
1	1	0	0	0
1	1	1	1	1

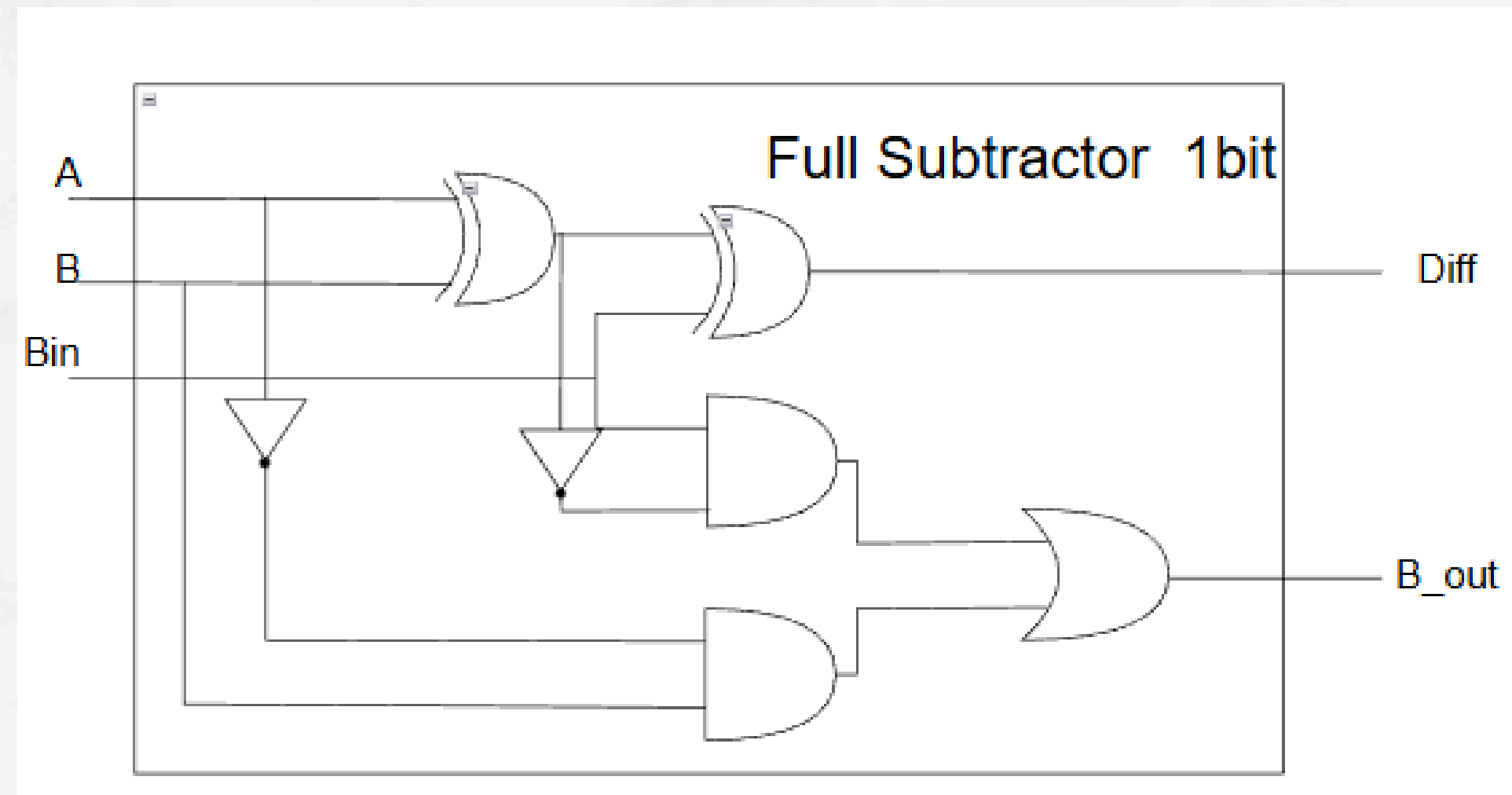
True Table of 1 bit Full Subtractor

2. DESIGN SPECIFICATION

2.3. Low Level Block Design

2.3.1. Arithmetic Unit Block

b. Full Subtractor



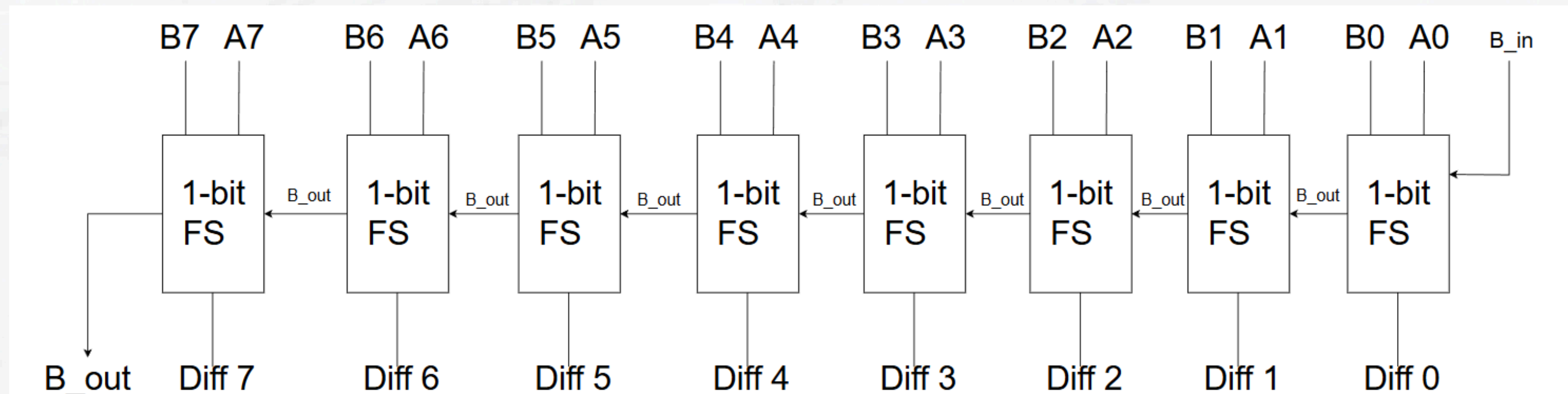
1 Bit Full Subtractor

2. DESIGN SPECIFICATION

2.3. Low Level Block Design

2.3.1. Arithmetic Unit Block

b. Ripper carry full Subtractor



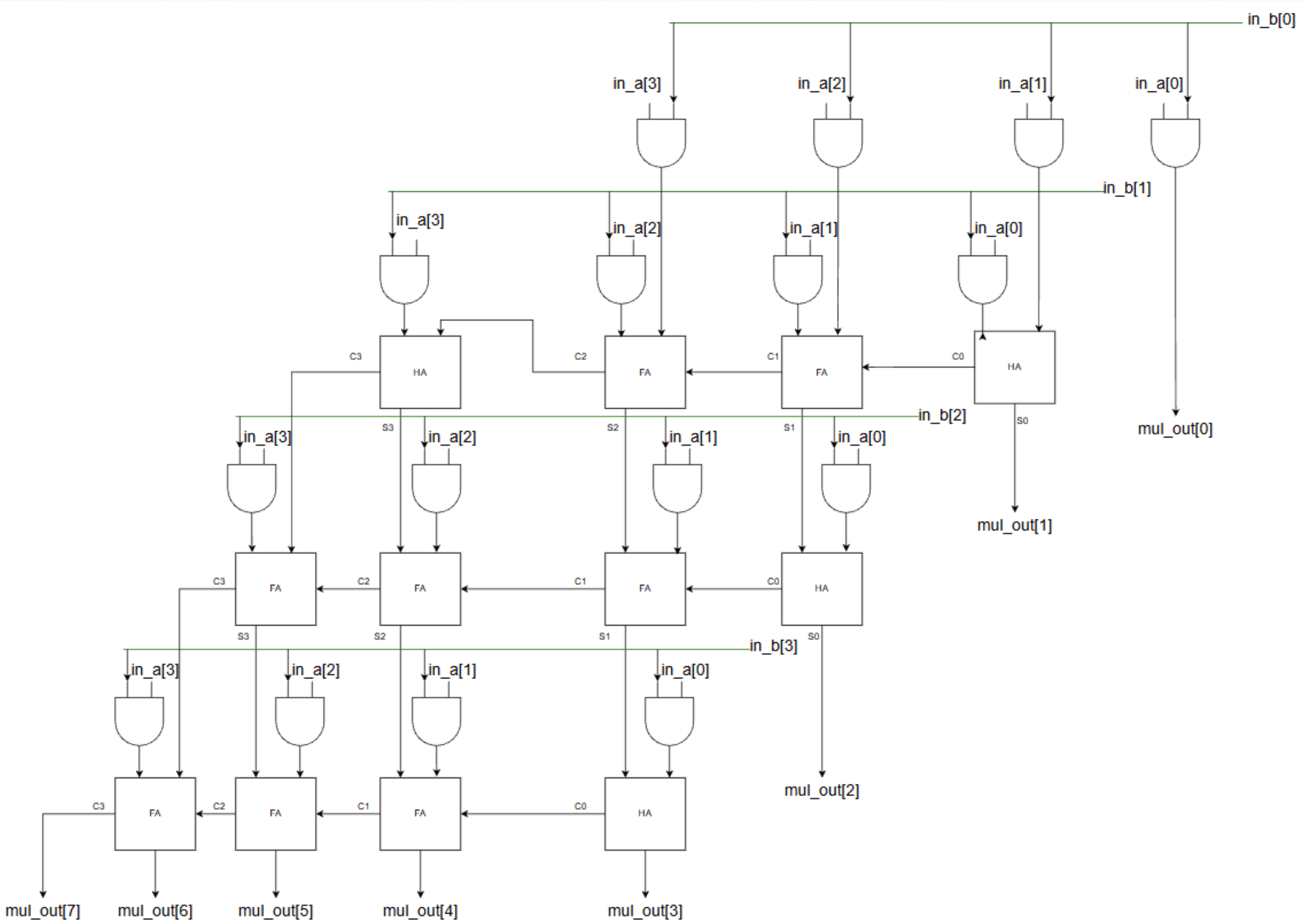
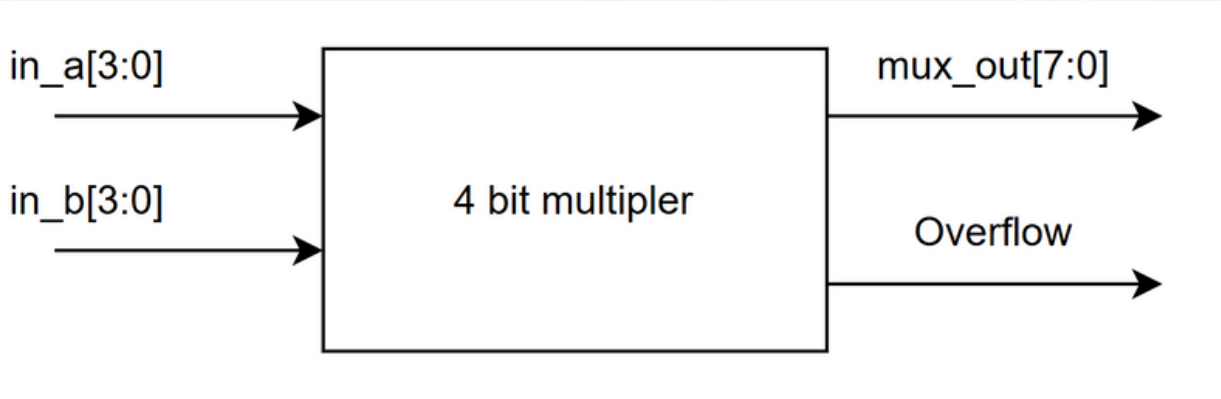
8 Bit Full Subtractor

2. DESIGN SPECIFICATION

2.3. Low Level Block Design

2.3.1. Arithmetic Unit Block

c. 4 Bit Array Mutiplier



2. DESIGN SPECIFICATION

2.3. Low Level Block Design

2.3.1. Arithmetic Unit Block

d. Comparator

Lối vào điều khiển			Lối vào dữ liệu		Lối ra		
c_3 a>b	c_2 a=b	c_1 a<b	a	b	y_3 a>b	y_2 a=b	y_1 a<b
1	0	0	x	x	1	0	0
0	0	1	x	x	0	0	1
0	1	0	0	0	0	1	0
0	1	0	0	1	0	0	1
0	1	0	1	0	1	0	0
0	1	0	1	1	0	1	0

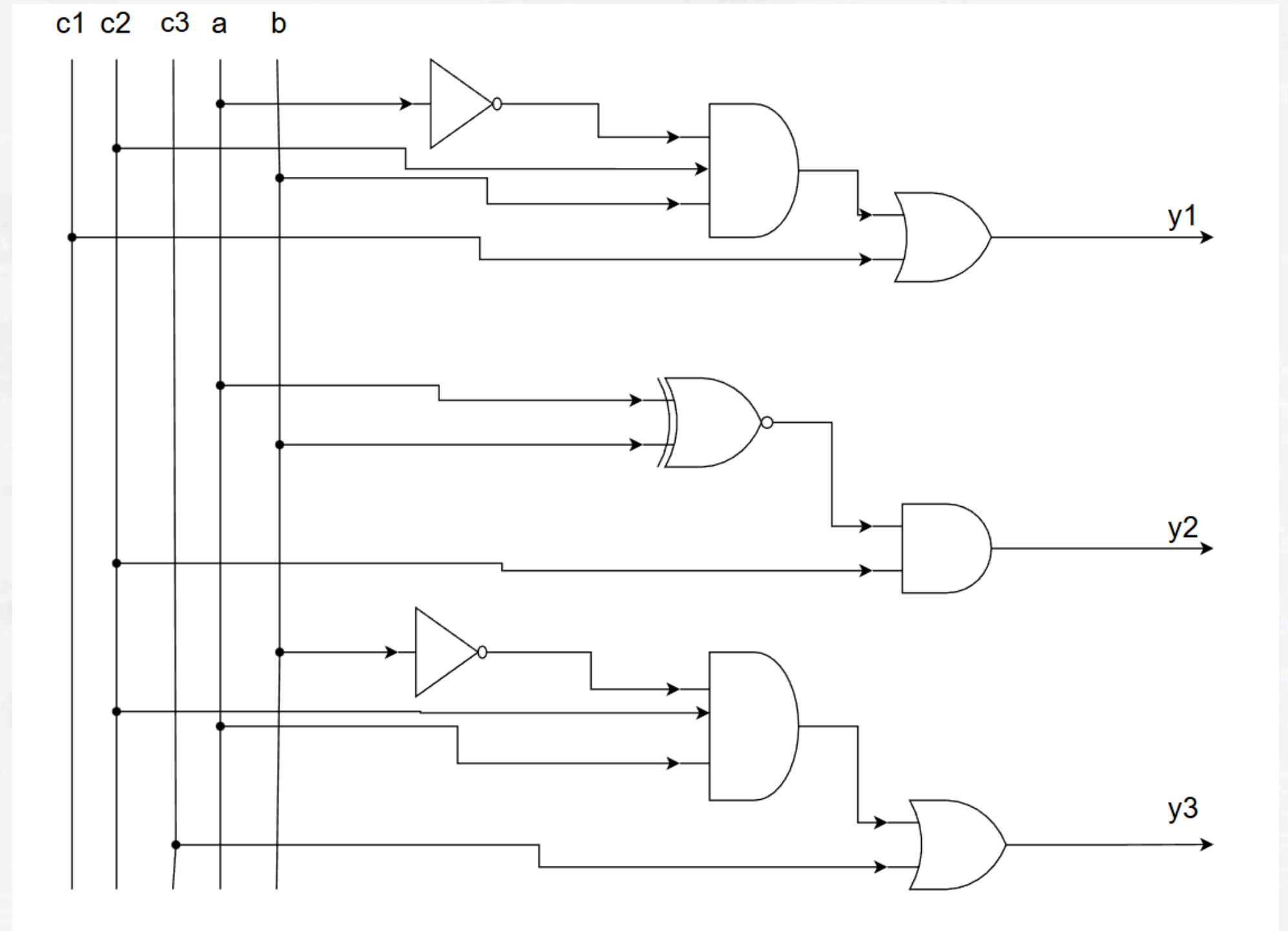
True Table of 1 bit Comparator

2. DESIGN SPECIFICATION

2.3. Low Level Block Design

2.3.1. Arithmetic Unit Block

d. Comparator



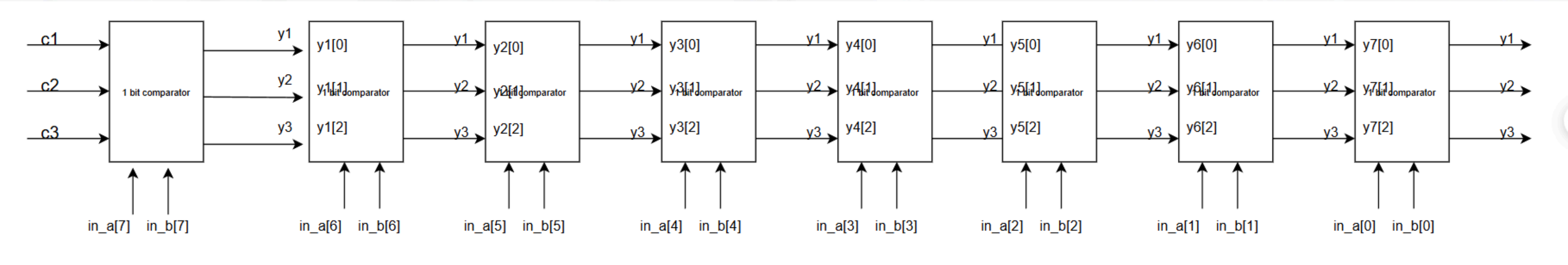
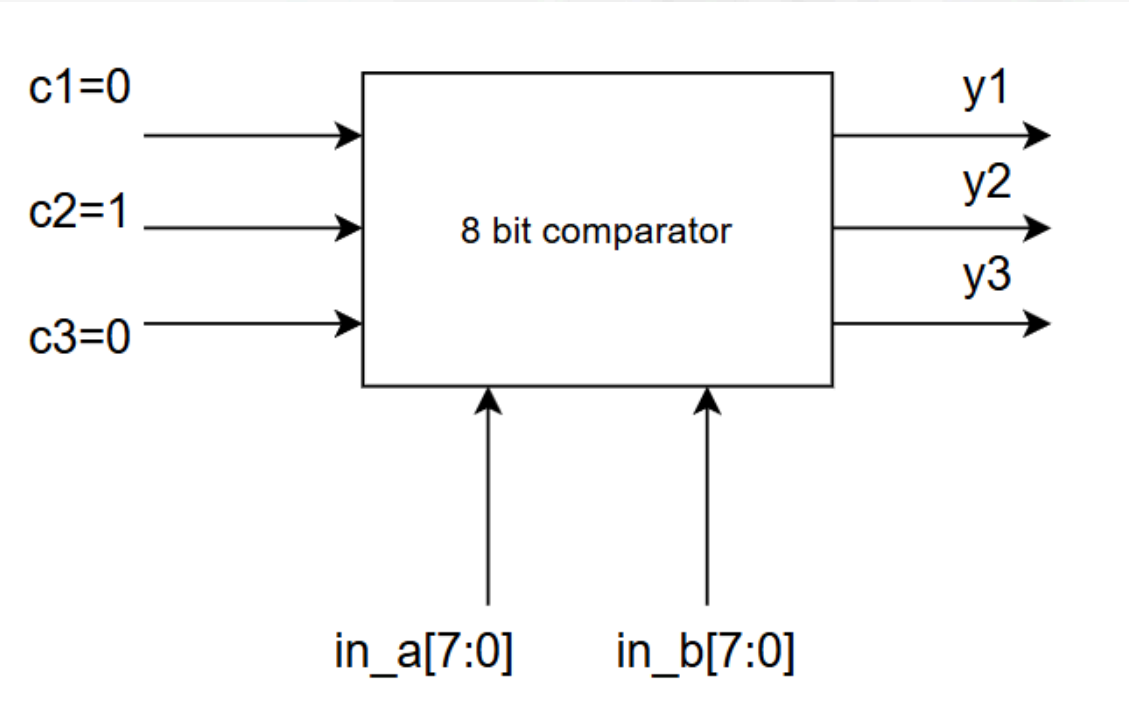
1 Bit Comparator

2. DESIGN SPECIFICATION

2.3. Low Level Block Design

2.3.1. Arithmetic Unit Block

d. Comparator



8 Bit Comparator

3. RTL DESCRIPTIONS

3. RTL DESCRIPTIONS

The following are a few examples of bitwise logical operations and basic arithmetic operations supported by ALUs:

- Addition: Adds A and B with carry-in or carry-out sum at Y.
- Subtraction: Subtracts B from A or vice versa with the difference at Y and carry-in or carry-out.
- Increment: Where A or B is increased by one and Y represents the new value.
- Decrement. Where A or B is decreased by one and Y represents the new value.
- AND: The bitwise logic AND of A and B is represented by Y.
- OR: The bitwise logic OR of A and B is represented by Y.
- Exclusive-OR: The bitwise logic XOR of A and B is represented by Y

```
1 module arith_unit#(parameter WIDTH=8)(
2     input [WIDTH-1:0] in_a,
3     input [WIDTH-1:0] in_b,
4     input input_carry,
5     input [2:0] alu_opcode,
6     output reg [WIDTH-1:0] arith_out,
7     output reg flag_mux_out
8 );
9     localparam [2:0] ADD =3'b000,
10     SUB =3'b001,
11     MUL =3'b010,
12     INC =3'b011,
13     DEC =3'b100,
14     SHIL =3'b101,
15     SHIR =3'b110;
16     wire [WIDTH-1:0] add_out,sub_out,mul_out,inc_out,dec_out,shift_out;
17     wire add_cf;
18     wire sub_cf;
```

```
1 `include "arith_unit.v"
2 `include "logic_unit.v"
3 module ALU#(parameter WIDTH=8)(
4     input [WIDTH-1:0] in_a,
5     input [WIDTH-1:0] in_b,
6     input input_carry,
7     input [3:0] alu_opcode,
8     output reg [WIDTH-1:0] alu_out,
9     output reg [4:0] alu_out_flag // carry_flag, over_flag,zero_flag,state_compare
10 );
11
12
13     localparam ZERO_INDEX=0;
14     localparam A_GT_B_INDEX=1;
15     localparam A_EQ_B_INDEX=2;
16     localparam A_LT_B_INDEX=3;
17     localparam FLAG_MUX_OUT_INDEX=4;//carry & overflow
18
```


3. RTL DESCRIPTIONS

```

1  `include "arith_unit.v"
2  `include "logic_unit.v"
3  module ALU#(parameter WIDTH=8)(
4      input [WIDTH-1:0] in_a,
5      input [WIDTH-1:0] in_b,
6      input input_carry,
7      input [3:0] alu_opcode,
8      output reg [WIDTH-1:0] alu_out,
9      output reg [4:0] alu_out_flag // c
10 );

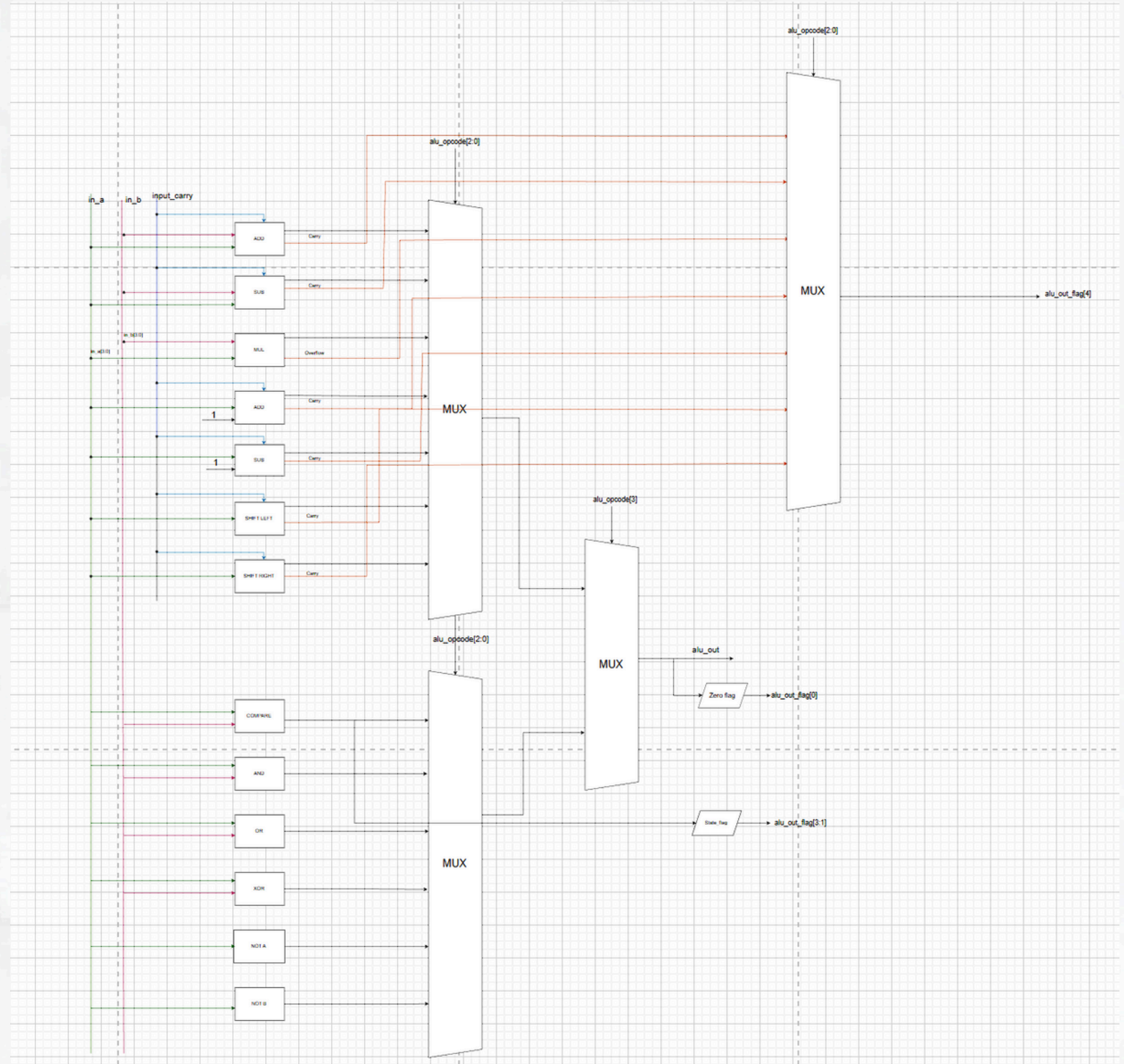
```

```

wire [WIDTH-1:0] arith_out;
wire [WIDTH-1:0] logic_out;
wire flag_mux_out;
wire w_gt, w_eq, w_lt;
arith_unit ari_module(
    .in_a(in_a),
    .in_b(in_b),
    .input_carry(input_carry),
    .alu_opcode(alu_opcode[2:0]),
    .arith_out(arith_out),
    .flag_mux_out(flag_mux_out)
);
logic_unit logic_module(
    .in_a(in_a),
    .in_b(in_b),
    .alu_opcode(alu_opcode[2:0]),
    .logic_out(logic_out),
    .comp_gt(w_gt),
    .comp_eq(w_eq),
    .comp_lt(w_lt)
);

```

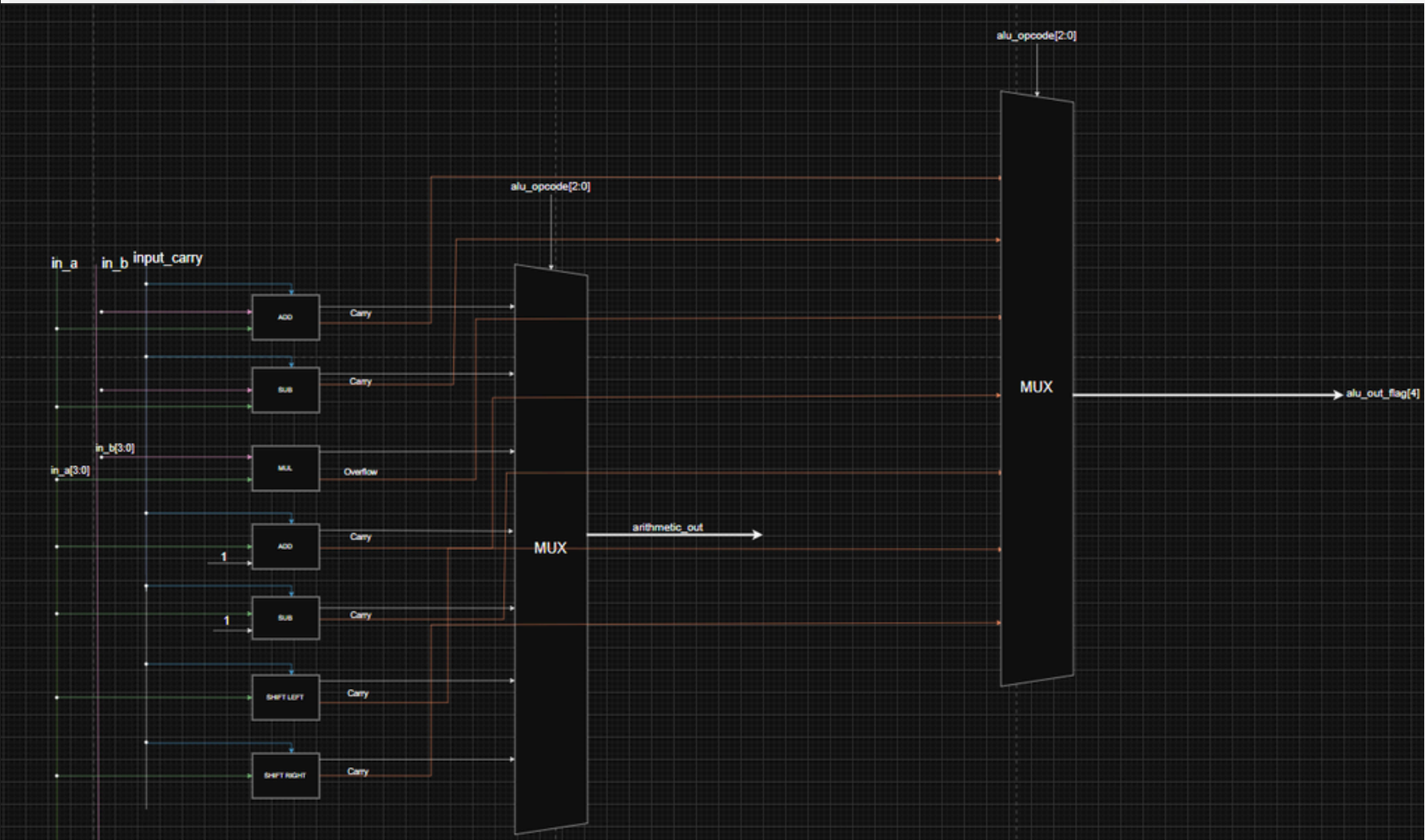
High Level Block Design



3. RTL DESCRIPTIONS

```
1 module arith_unit#(parameter WIDTH=8)(
2     input [WIDTH-1:0] in_a,
3     input [WIDTH-1:0] in_b,
4     input input_carry,
5     input [2:0] alu_opcode,
6     output reg [WIDTH-1:0] arith_out,
7     output reg flag_mux_out
8 );
9 localparam [2:0]  ADD =3'b000,
10                  SUB =3'b001,
11                  MUL =3'b010,
12                  INC =3'b011,
13                  DEC =3'b100,
14                  SHIL =3'b101,
15                  SHIR =3'b110;
16 wire [WIDTH-1:0] add_out,sub_out,mul_out,inc_out,dec_out,shift_out;
17 wire add_cf;
18 wire sub_cf;
19 wire mul_of;
20 wire inc_cf;
21 wire dec_cf;
22 wire shift_cf;
23 fadder_8bit ADD_unit(
24     .a(in_a),
25     .b(in_b),
26     .cin(input_carry),
27     .s(add_out),
28     .cout(add_cf)
29 );
30 fsubtrator_8bit SUB_unit(
31     .a(in_a),
32     .b(in_b),
33     .bin(input_carry),
34     .d(sub_out),
35     .bout(sub_cf)
36 );
```

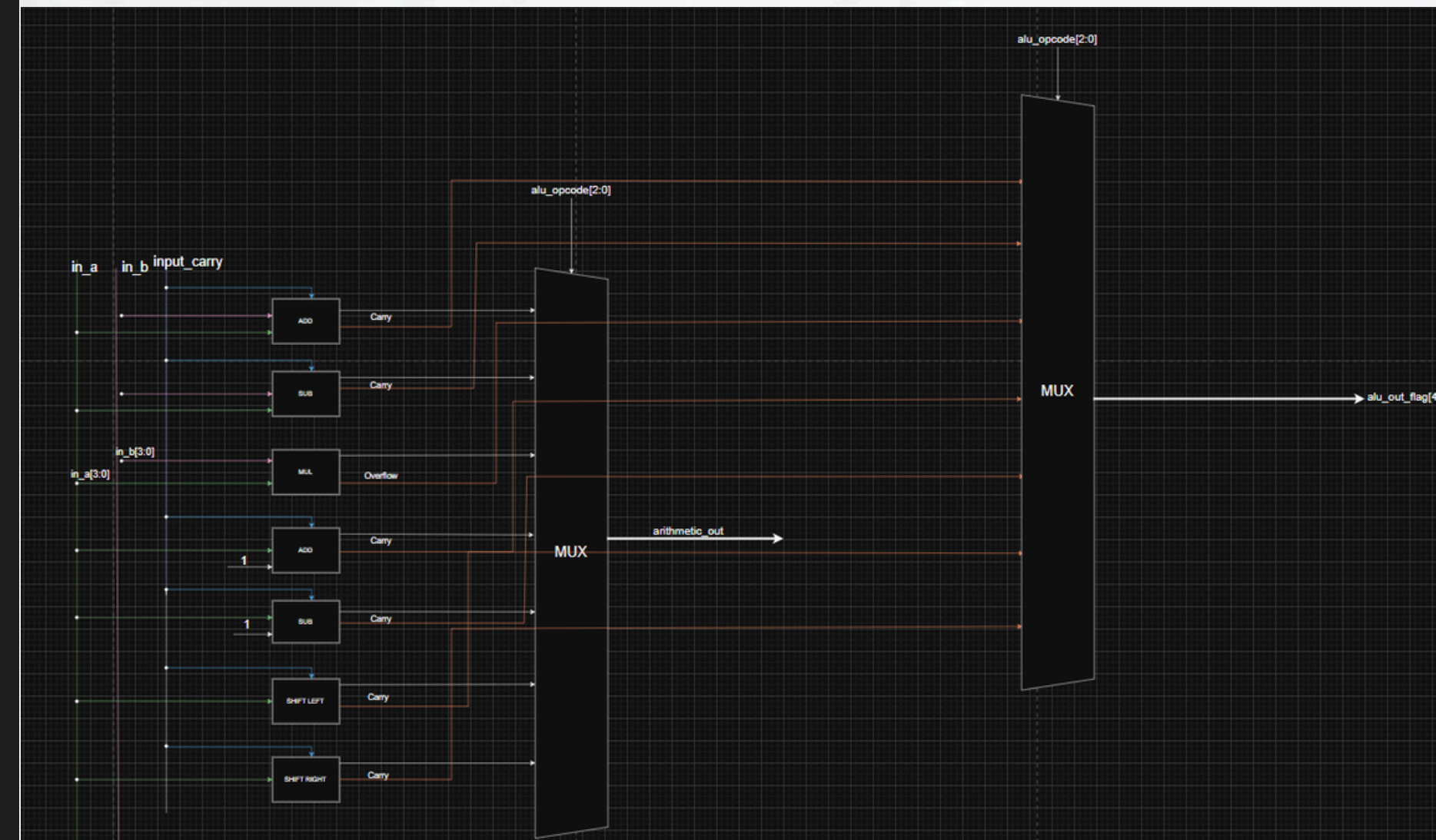
Arith Unit Block



3. RTL DESCRIPTIONS

```
37 multiply MUL_unit(  
38     .in_a(in_a[3:0]),  
39     .in_b(in_b[3:0]),  
40     .mul_out(mul_out),  
41     .overflow(mul_of)  
42 );  
43 fadder_8bit inc_unit(  
44     .a(in_a),  
45     .b(8'd1),  
46     .cin(input_carry),  
47     .s(inc_out),  
48     .cout(inc_cf)  
49 );  
50 fsubtrator_8bit dec_unit(  
51     .a(in_a),  
52     .b(8'd1),  
53     .bin(input_carry),  
54     .d(dec_out),  
55     .bout(dec_cf)  
56 );  
57 shifter_8bit SHIFT_unit(  
58     .in_a(in_a),  
59     .input_carry(input_carry),  
60     .opcode(alu_opcode),  
61     .shift_out(shift_out),  
62     .shift_cf(shift_cf)  
63 );
```

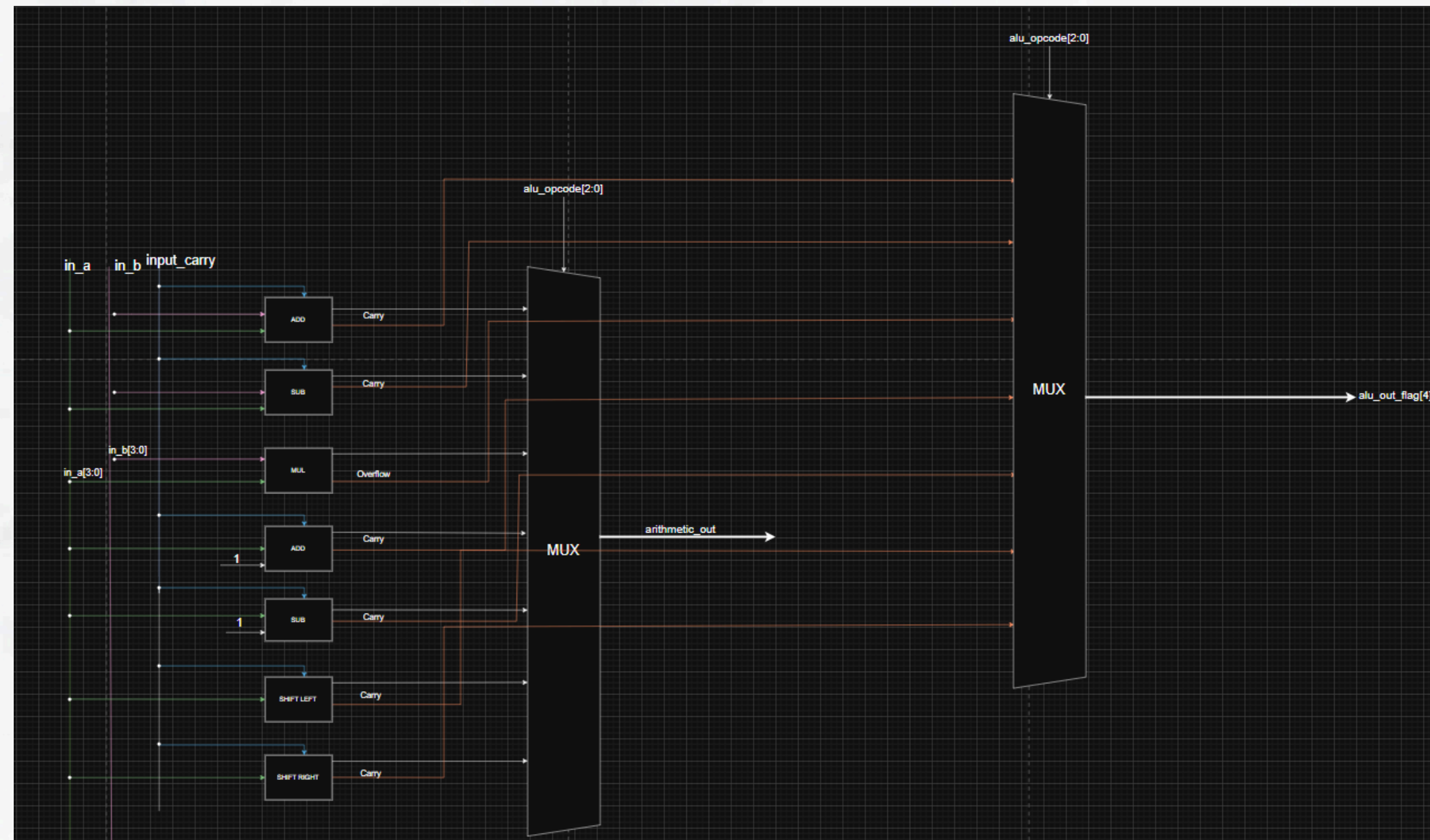
Arith Unit Block



3. RTL DESCRIPTIONS

Arith Unit Block

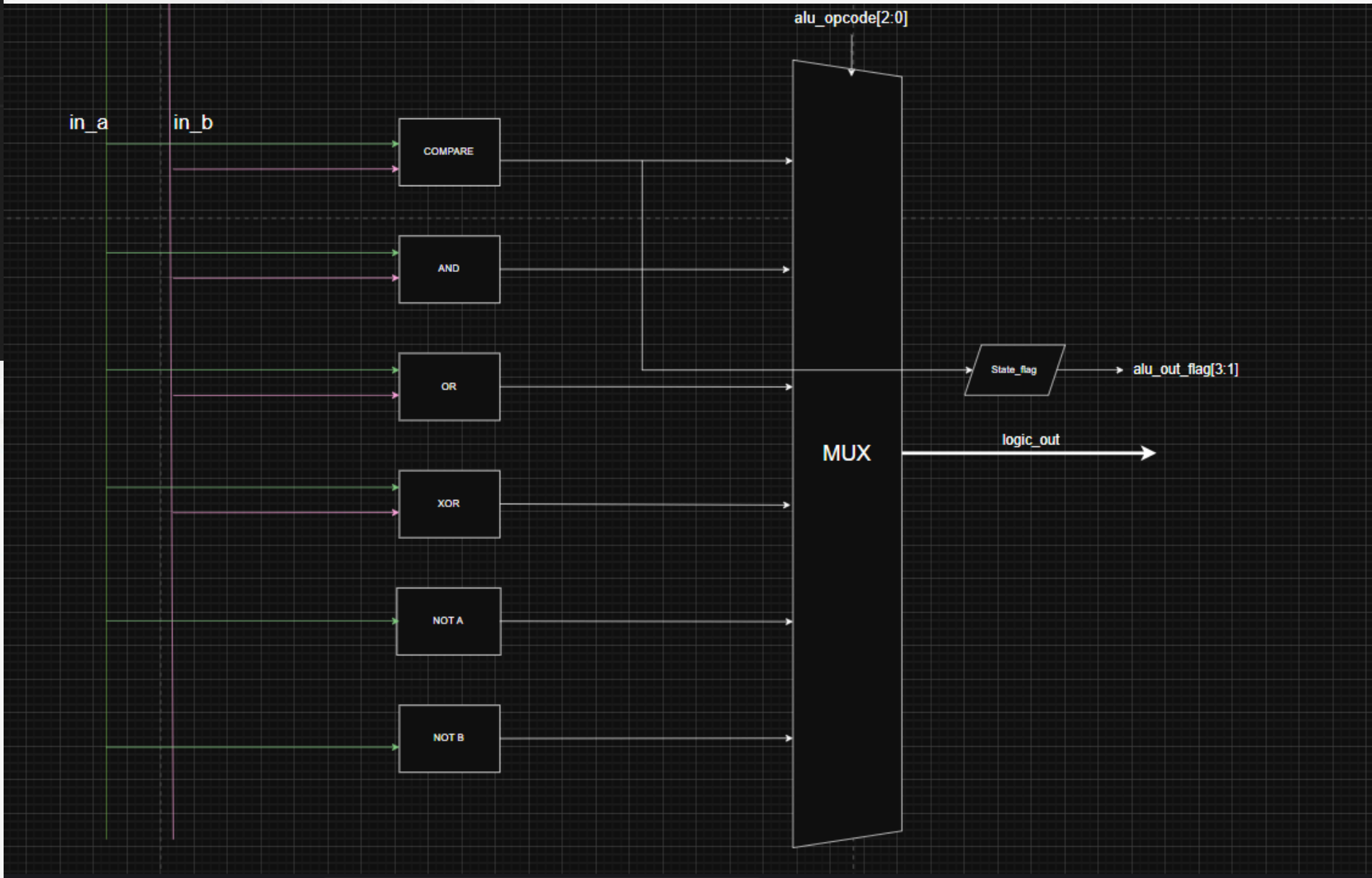
```
64 always@(*)begin
65     arith_out=0;
66     flag_mux_out=0;
67     case(alu_opcode[2:0])
68     ADD:begin
69         arith_out=add_out;
70         flag_mux_out=add_cf;
71     end
72     SUB:begin
73         arith_out=sub_out;
74         flag_mux_out=sub_cf;
75     end
76     MUL:begin
77         arith_out=mul_out;
78         flag_mux_out=mul_of;
79     end
80     INC:begin
81         arith_out=inc_out;
82         flag_mux_out=add_cf;
83     end
84     DEC:begin
85         arith_out=dec_out;
86         flag_mux_out=sub_cf;
87     end
88     SHIL,SHIR:begin
89         arith_out=shift_out;
90         flag_mux_out=shift_cf;
91     end
92     endcase
93 end
94 endmodule
```



3. RTL DESCRIPTIONS

```
1 module logic_unit#(parameter WIDTH=8)(
2     input [WIDTH-1:0] in_a,
3     input [WIDTH-1:0] in_b,
4     input [2:0] alu_opcode,
5     output reg [WIDTH-1:0] logic_out,
6     output comp_gt,
7     output comp_eq,
8     output comp_lt
9 );
10 localparam [2:0] COMP=3'b000,ANDG=3'b001,ORG=3'b010,XORG=3'b011,NOTA=3'b100,NOTB=3'b101;
11 wire [2:0] comp_result;
12 compare_8bit my_comparator(
13     .in_a(in_a),
14     .in_b(in_b),
15     .y(comp_result)
16 );
17 assign comp_lt=comp_result[0];
18 assign comp_eq=comp_result[1];
19 assign comp_gt=comp_result[2];
20 always@(*)begin
21     case(alu_opcode)
22     COMP:begin
23         logic_out=(comp_result[1]) ? 1'b1:1'b0;
24     end
25     ANDG:begin
26         logic_out=in_a & in_b;
27     end
28     ORG:begin
29         logic_out=in_a | in_b;
30     end
31     XORG:begin
32         logic_out=in_a ^ in_b;
33     end
34     NOTA:begin
35         logic_out=~in_a;
36     end
37     NOTB:begin
38         logic_out=~in_b;
39     end
40     default:logic_out=0;
41     endcase
42 end
43 endmodule
```

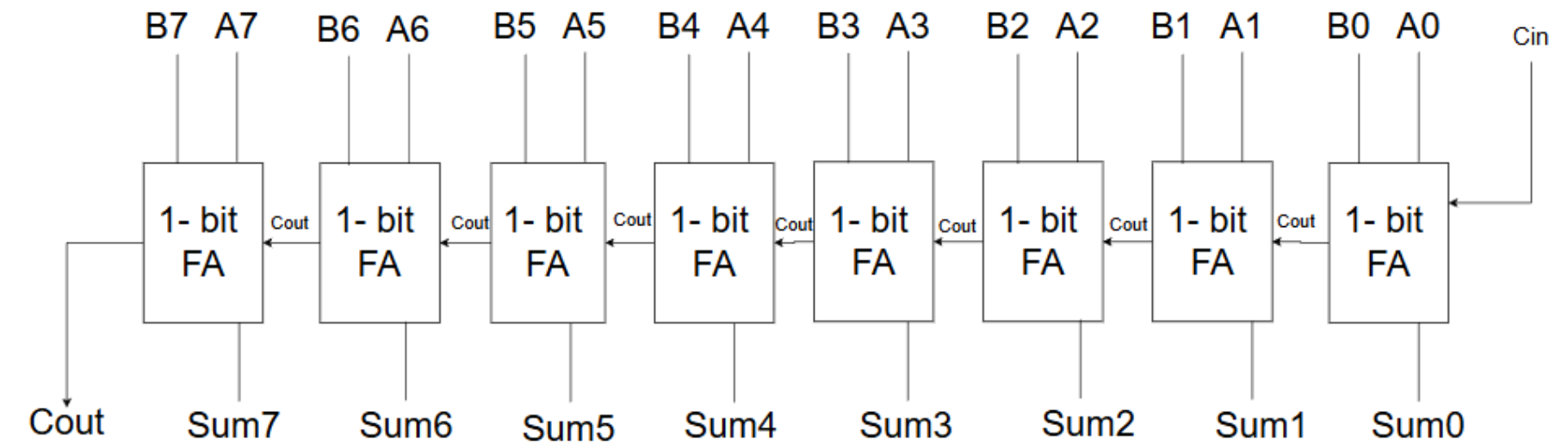
Logic Unit Block



3. RTL DESCRIPTIONS

8 Bit Full Adder

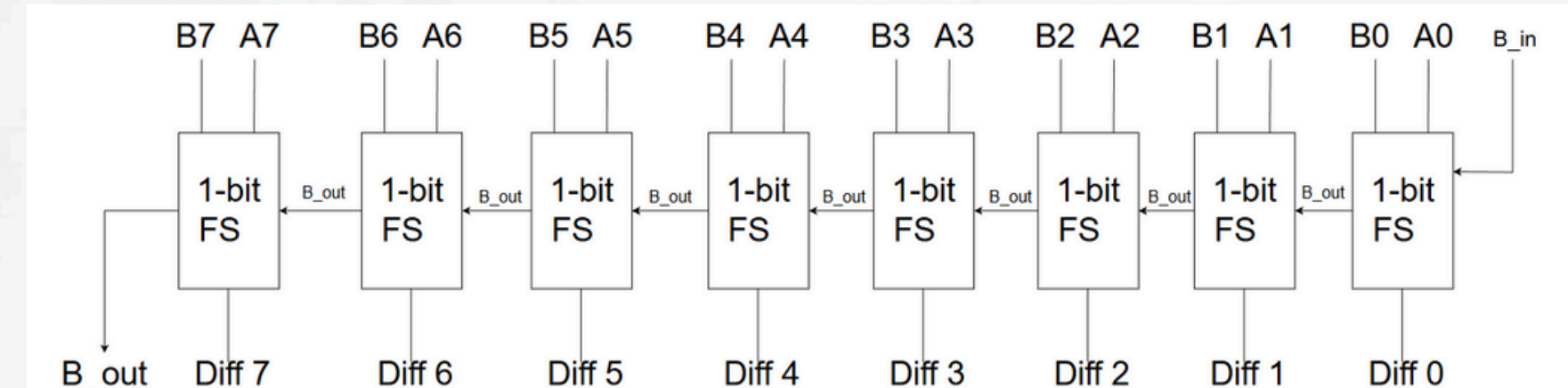
```
96  module fadder_1bit(  
97      input a,  
98      input b,  
99      input cin,  
100     output s,  
101     output cout  
102 );  
103     assign s=a^b^cin;  
104     assign cout=(a&b)|(cin & (a^b));  
105 endmodule  
106  
107  module fadder_8bit#(parameter WIDTH=8)(  
108      input [WIDTH-1:0] a,  
109      input [WIDTH-1:0] b,  
110      input cin,  
111      output [WIDTH-1:0] s,  
112      output cout,  
113  );  
114      wire cin1,cin2,cin3,cin4,cin5,cin6,cin7;  
115  
116      fadder_1bit F0(.a(a[0]),.b(b[0]),.cin(cin),.s(s[0]),.cout(cin1));  
117      fadder_1bit F1(.a(a[1]),.b(b[1]),.cin(cin1),.s(s[1]),.cout(cin2));  
118      fadder_1bit F2(.a(a[2]),.b(b[2]),.cin(cin2),.s(s[2]),.cout(cin3));  
119      fadder_1bit F3(.a(a[3]),.b(b[3]),.cin(cin3),.s(s[3]),.cout(cin4));  
120      fadder_1bit F4(.a(a[4]),.b(b[4]),.cin(cin4),.s(s[4]),.cout(cin5));  
121      fadder_1bit F5(.a(a[5]),.b(b[5]),.cin(cin5),.s(s[5]),.cout(cin6));  
122      fadder_1bit F6(.a(a[6]),.b(b[6]),.cin(cin6),.s(s[6]),.cout(cin7));  
123      fadder_1bit F7(.a(a[7]),.b(b[7]),.cin(cin7),.s(s[7]),.cout(cout));  
124 endmodule  
125
```



3. RTL DESCRIPTIONS

8 Bit Full Subtractor

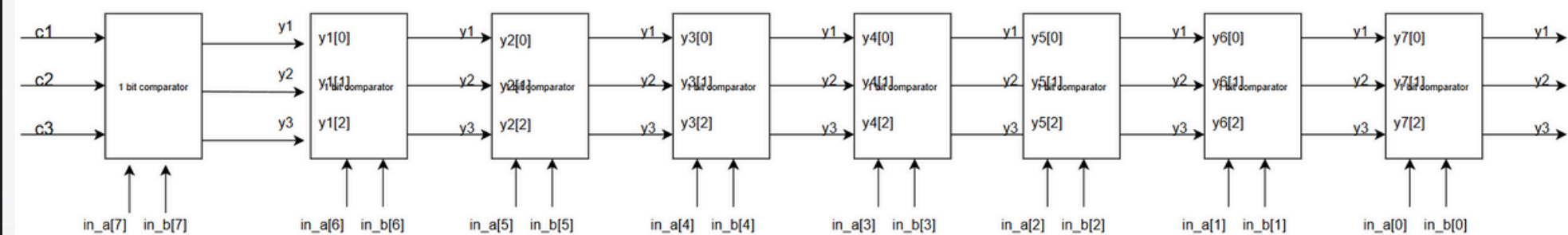
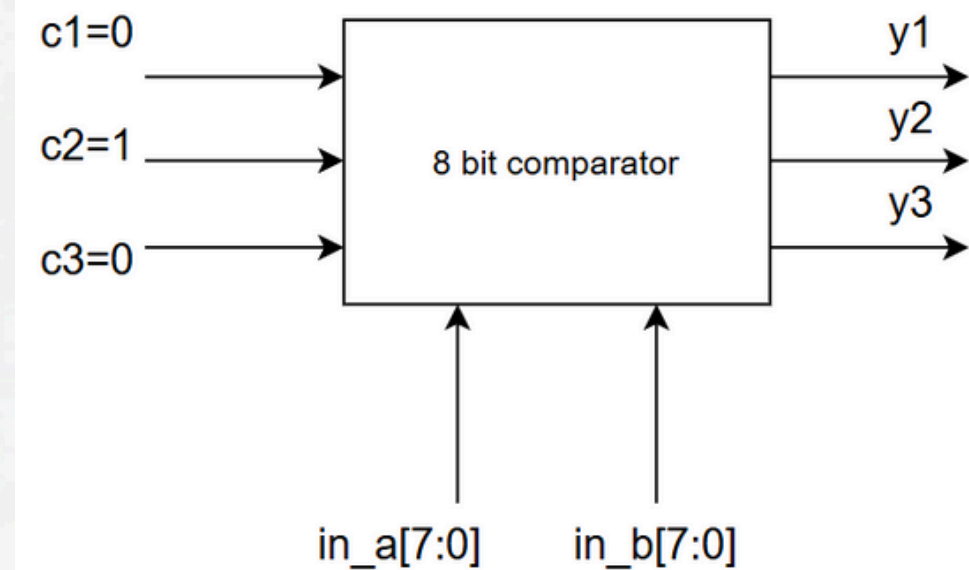
```
126 module fsubtrator_1bit(  
127     input a,  
128     input b,  
129     input bin,  
130     output d,  
131     output bout  
132 );  
133     assign d=a^b^bin;  
134     assign bout=~a & b | (bin&(~(a^b)));  
135 endmodule  
136  
137 module fsubtrator_8bit#(parameter WIDTH=8)(  
138     input [WIDTH-1:0] a,  
139     input [WIDTH-1:0] b,  
140     input bin,  
141     output[WIDTH-1:0] d,  
142     output bout,  
143 );  
144     wire bin1,bin2,bin3,bin4,bin5,bin6,bin7;  
145     fsubtrator_1bit D0(.a(a[0]),.b(b[0]),.bin(bin),.d(d[0]),.bout(bin1));  
146     fsubtrator_1bit D1(.a(a[1]),.b(b[1]),.bin(bin1),.d(d[1]),.bout(bin2));  
147     fsubtrator_1bit D2(.a(a[2]),.b(b[2]),.bin(bin2),.d(d[2]),.bout(bin3));  
148     fsubtrator_1bit D3(.a(a[3]),.b(b[3]),.bin(bin3),.d(d[3]),.bout(bin4));  
149     fsubtrator_1bit D4(.a(a[4]),.b(b[4]),.bin(bin4),.d(d[4]),.bout(bin5));  
150     fsubtrator_1bit D5(.a(a[5]),.b(b[5]),.bin(bin5),.d(d[5]),.bout(bin6));  
151     fsubtrator_1bit D6(.a(a[6]),.b(b[6]),.bin(bin6),.d(d[6]),.bout(bin7));  
152     fsubtrator_1bit D7(.a(a[7]),.b(b[7]),.bin(bin7),.d(d[7]),.bout(bout));  
153 endmodule
```



3. RTL DESCRIPTIONS

```
45 module compare_1bit(  
46     input a,  
47     input b,  
48     input c1,c2,c3,  
49     output [2:0] y  
50 );  
51 assign y[0]= c1 | c2&(~a &b); //lt  
52 assign y[1]=c2 & ~(a^b); //eq  
53 assign y[2]=c3 | c2 & (a&~b); //grt  
54 endmodule  
55  
56 module compare_8bit#(parameter WIDTH=8)(  
57     input [WIDTH-1:0] in_a,  
58     input [WIDTH-1:0] in_b,  
59     output [2:0] y  
60 );  
61 wire [2:0] y1;  
62 wire [2:0] y2;  
63 wire [2:0] y3;  
64 wire [2:0] y4;  
65 wire [2:0] y5;  
66 wire [2:0] y6;  
67 wire [2:0] y7;  
68 compare_1bit F0(.a(in_a[7]),.b(in_b[7]),.c1(1'b0),.c2(1'b1),.c3(1'b0),.y(y1));  
69 compare_1bit F1(.a(in_a[6]),.b(in_b[6]),.c1(y1[0]),.c2(y1[1]),.c3(y1[2]),.y(y2));  
70 compare_1bit F2(.a(in_a[5]),.b(in_b[5]),.c1(y2[0]),.c2(y2[1]),.c3(y2[2]),.y(y3));  
71 compare_1bit F3(.a(in_a[4]),.b(in_b[4]),.c1(y3[0]),.c2(y3[1]),.c3(y3[2]),.y(y4));  
72 compare_1bit F4(.a(in_a[3]),.b(in_b[3]),.c1(y4[0]),.c2(y4[1]),.c3(y4[2]),.y(y5));  
73 compare_1bit F5(.a(in_a[2]),.b(in_b[2]),.c1(y5[0]),.c2(y5[1]),.c3(y5[2]),.y(y6));  
74 compare_1bit F6(.a(in_a[1]),.b(in_b[1]),.c1(y6[0]),.c2(y6[1]),.c3(y6[2]),.y(y7));  
75 compare_1bit F7(.a(in_a[0]),.b(in_b[0]),.c1(y7[0]),.c2(y7[1]),.c3(y7[2]),.y(y));  
76 endmodule
```

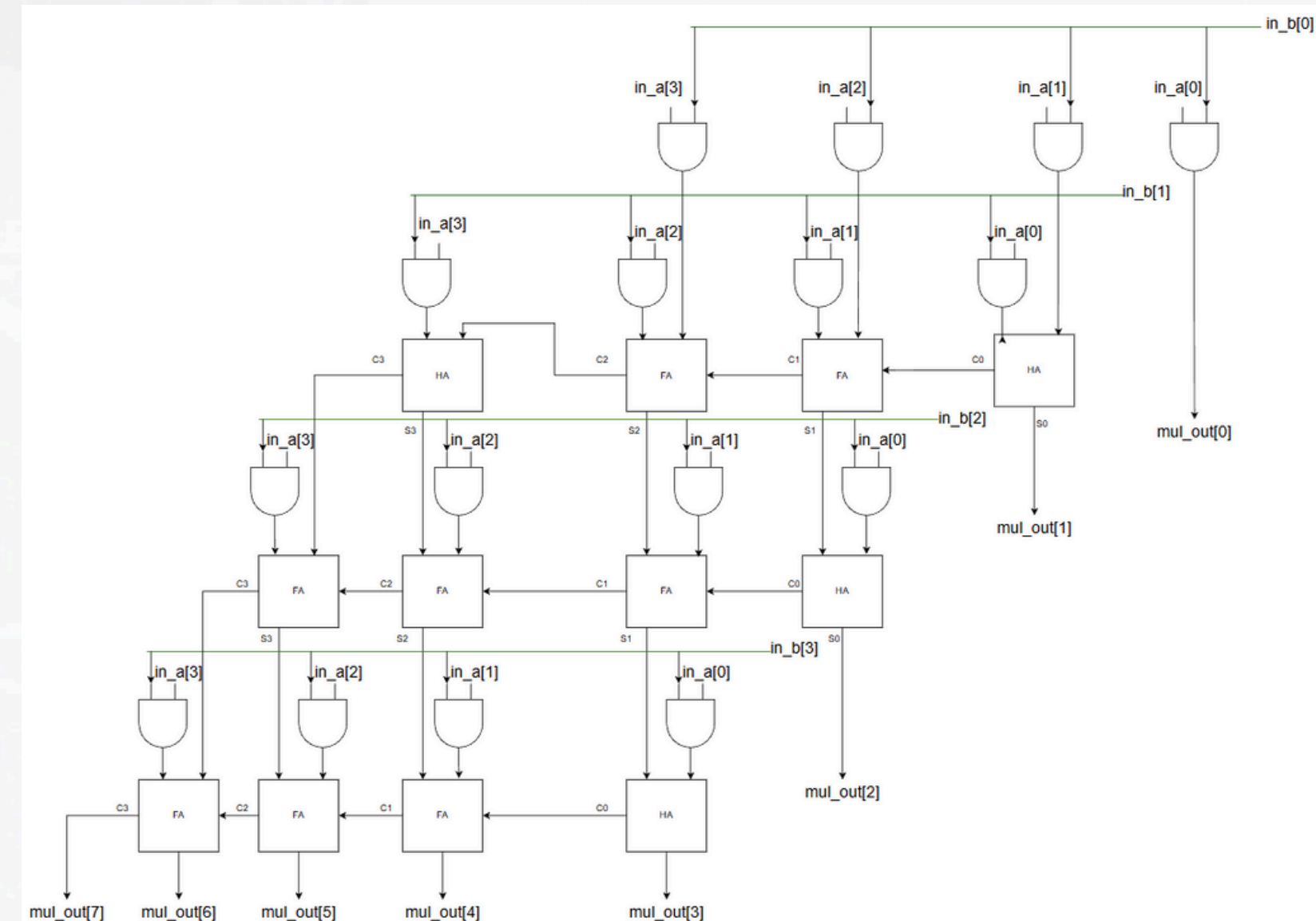
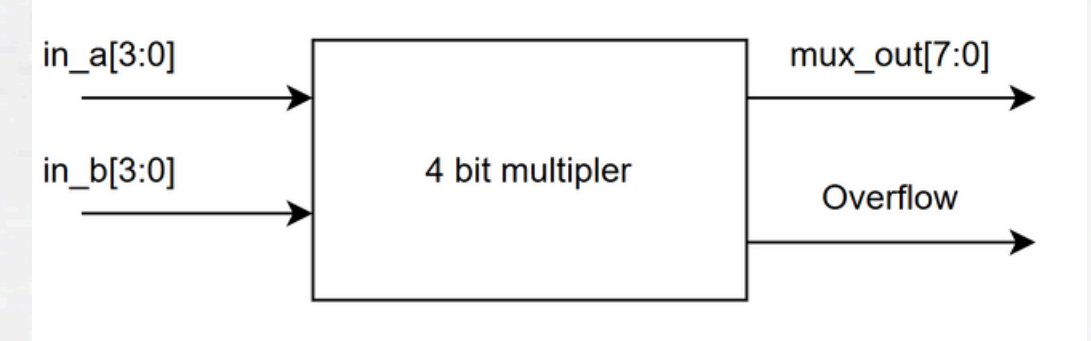
8 Bit Comparator



3. RTL DESCRIPTIONS

```
155 module multiply(  
156     input [3:0] in_a,  
157     input [3:0] in_b,  
158     output [7:0] mul_out,  
159     output overflow  
160 );  
161 wire [3:0] s_r1,c_r1;// hang 1  
162 wire [3:0] s_r2,c_r2;// hang 2  
163 wire [3:0] s_r3,c_r3;// hang 3  
164 assign mul_out[0] = in_a[0] & in_b[0];  
165 //HANG 1  
166 half_adder_1bit H0_r1(.a(in_a[1]&in_b[0]),.b(in_a[0]&in_b[1]),.s(mul_out[1]),.cout(c_r1[0]));  
167 fadder_1bit H1_r1(.a(in_a[2]&in_b[0]),.b(in_a[1]&in_b[1]),.cin(c_r1[0]),.s(s_r1[1]),.cout(c_r1[1]));  
168 fadder_1bit H2_r1(.a(in_a[3]&in_b[0]),.b(in_a[2]&in_b[1]),.cin(c_r1[1]),.s(s_r1[2]),.cout(c_r1[2]));  
169 half_adder_1bit H3_r1(.a(in_a[3]&in_b[1]),.b(c_r1[2]),.s(s_r1[3]),.cout(c_r1[3]));  
170 //HANG 2  
171 half_adder_1bit H4_r2(.a(in_a[0]&in_b[2]),.b(s_r1[1]),.s(mul_out[2]),.cout(c_r2[0]));  
172 fadder_1bit H5_r2(.a(in_a[1]&in_b[2]),.b(s_r1[2]),.cin(c_r2[0]),.s(s_r2[1]),.cout(c_r2[1]));  
173 fadder_1bit H6_r2(.a(in_a[2]&in_b[2]),.b(s_r1[3]),.cin(c_r2[1]),.s(s_r2[2]),.cout(c_r2[2]));  
174 fadder_1bit H7_r2(.a(in_a[3]&in_b[2]),.b(c_r1[3]),.cin(c_r2[2]),.s(s_r2[3]),.cout(c_r2[3]));  
175 //HANG 3  
176 half_adder_1bit H8_r3(.a(in_a[0]&in_b[3]),.b(s_r2[1]),.s(mul_out[3]),.cout(c_r3[0]));  
177 fadder_1bit H9_r3(.a(in_a[1]&in_b[3]),.b(s_r2[2]),.cin(c_r3[0]),.s(mul_out[4]),.cout(c_r3[1]));  
178 fadder_1bit H10_r3(.a(in_a[2]&in_b[3]),.b(s_r2[3]),.cin(c_r3[1]),.s(mul_out[5]),.cout(c_r3[2]));  
179 fadder_1bit H11_r3(.a(in_a[3]&in_b[3]),.b(c_r2[3]),.cin(c_r3[2]),.s(mul_out[6]),.cout(mul_out[7]));  
180 assign overflow=mul_out[7] | mul_out[6] | mul_out[5] | mul_out[4];  
181 endmodule  
182  
183 module half_adder_1bit(  
184     input a,  
185     input b,  
186     output s,  
187     output cout  
188 );  
189     assign s=a^b;  
190     assign cout=a&b;  
191 endmodule
```

4 Bit Multiplier

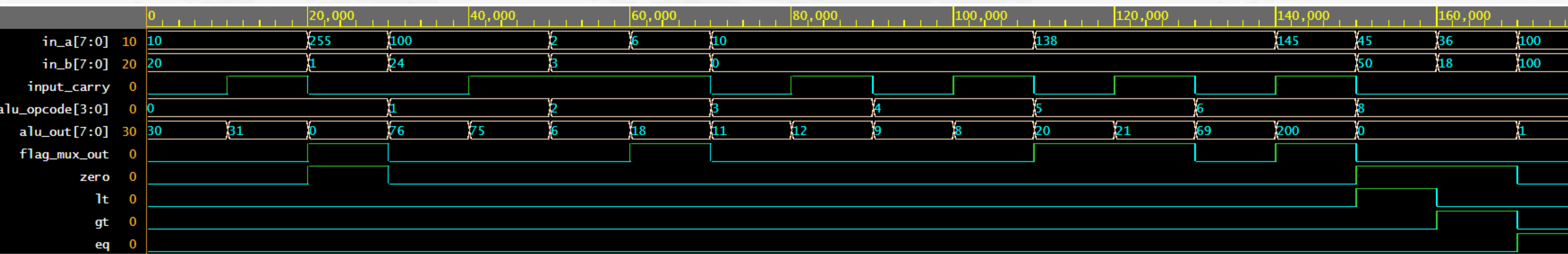


3. RTL DESCRIPTIONS

link to EDA playground:8 bit ALU

4. RESULT AND SIMULATIONS

4.RESULT AND SIMULATIONS



To revert to EDAWave opening in a new browser window, set that option on your profile page.

4. RESULT AND SIMULATIONS

```
=====TEST ARITHMETIC=====
ADD: 10 + 20
time =          0 | a = 10(a=00001010) | b = 20(b=00010100) | input_carry = 0 | alu_opcode =0000 ,| alu_out = 30(alu_out=00011110) | alu_out_flag = 00000{flag_mux_out(cari
ADD WITH CARRY: 10 + 20 +1
time =         10000 | a = 10(a=00001010) | b = 20(b=00010100) | input_carry = 1 | alu_opcode =0000 ,| alu_out = 31(alu_out=00011111) | alu_out_flag = 00000{flag_mux_out(cari
TEST CARRY FLAG: 255+1
time =         20000 | a =255(a=11111111) | b = 1(b=00000001) | input_carry = 0 | alu_opcode =0000 ,| alu_out = 0(alu_out=00000000) | alu_out_flag = 10001{flag_mux_out(cari
SUB: 100 - 24
time =         30000 | a =100(a=01100100) | b = 24(b=00011000) | input_carry = 0 | alu_opcode =0001 ,| alu_out = 76(alu_out=01001100) | alu_out_flag = 00000{flag_mux_out(cari
SUB WITH CARRY:100 -24 -1
time =         40000 | a =100(a=01100100) | b = 24(b=00011000) | input_carry = 1 | alu_opcode =0001 ,| alu_out = 75(alu_out=01001011) | alu_out_flag = 00000{flag_mux_out(cari
MUL: 2 * 3
time =         50000 | a = 2(a=00000010) | b = 3(b=00000011) | input_carry = 1 | alu_opcode =0010 ,| alu_out = 6(alu_out=00000110) | alu_out_flag = 00000{flag_mux_out(cari
MUL OVERFLOW FLAG: 6 *3
time =         60000 | a = 6(a=00000110) | b = 3(b=00000011) | input_carry = 1 | alu_opcode =0010 ,| alu_out = 18(alu_out=00010010) | alu_out_flag = 10000{flag_mux_out(c
INCREASE: 10 + 1
time =         70000 | a = 10(a=00001010) | b = 0(b=00000000) | input_carry = 0 | alu_opcode =0011 ,| alu_out = 11(alu_out=00001011) | alu_out_flag = 00000{flag_mux_out(cari
INCREASE WITH CARRY: 10 +1 +1
time =         80000 | a = 10(a=00001010) | b = 0(b=00000000) | input_carry = 1 | alu_opcode =0011 ,| alu_out = 12(alu_out=00001100) | alu_out_flag = 00000{flag_mux_out(cari
DECREASE: 10 -1
time =         90000 | a = 10(a=00001010) | b = 0(b=00000000) | input_carry = 0 | alu_opcode =0100 ,| alu_out = 9(alu_out=00001001) | alu_out_flag = 00000{flag_mux_out(cari
DECREASE WITH CARRY: 10 - 1 -1
time =        100000 | a = 10(a=00001010) | b = 0(b=00000000) | input_carry = 1 | alu_opcode =0100 ,| alu_out = 8(alu_out=00001000) | alu_out_flag = 00000{flag_mux_out(cari
SHIFT LEFT
time =        110000 | a =138(a=10001010) | b = 0(b=00000000) | input_carry = 0 | alu_opcode =0101 ,| alu_out = 20(alu_out=00010100) | alu_out_flag = 10000{flag_mux_out(cari
SHIFT LEFT WITH CARRY
time =        120000 | a =138(a=10001010) | b = 0(b=00000000) | input_carry = 1 | alu_opcode =0101 ,| alu_out = 21(alu_out=00010101) | alu_out_flag = 10000{flag_mux_out(cari
SHIFT RIGHT
time =        130000 | a =138(a=10001010) | b = 0(b=00000000) | input_carry = 0 | alu_opcode =0110 ,| alu_out = 69(alu_out=01000101) | alu_out_flag = 00000{flag_mux_out(cari
SHIFT RIGHT WITH CARRY
time =        140000 | a =145(a=10010001) | b = 0(b=00000000) | input_carry = 1 | alu_opcode =0110 ,| alu_out =200(alu_out=11001000) | alu_out_flag = 10000{flag_mux_out(cari
-----TEST LOGIC-----
```

4. RESULT AND SIMULATIONS

Group: in2out

```
Startpoint: (R) in_b[0]
```

Clock: (R) vclk

```
Endpoint: (R) alu_out_flag[0]
```

Clock: (R) vclk

Capture

Launch

Clock Edge: +

500

0

Drv Adjust:+

0

0

Src Latency:+

0

0

```
Net Latency:+      0 (I)
```

0 (I)

Arrival:=

500

0

Output Delay:-

100

Required Time:=

400

Launch Clock:-

0

Input Delay:-

100

Data Path:-

274

Slack:=

26

Exceptions/Constraints:

```
input_delay      100
```

```
ALU_sdc.sdc_line_25_13_1
```

```
output_delay      100
```

ALU_sdc.sdc_line_44_24_1

#	Timing Point	Flags	Arc	Edge	Cell	Fanout	Load (fF)	Trans (ps)	Delay (ps)	Arrival (ps)	Instance Location
#	in_b[0]	-	-	R	(arrival)	9	2.7	0	0	100	(-, -)
	g12725__2398/Y	-	A->Y	F	NAND2XL	4	1.2	22	16	116	(-, -)
	g12817/Y	-	B->Y	R	NOR3BXL	2	0.6	26	24	141	(-, -)
	g12624/Y	-	A->Y	F	INVXL	2	0.8	12	13	154	(-, -)
	g12563__2883/Y	-	A1->Y	R	OAI22X1	5	1.8	37	24	178	(-, -)
	g12538__7410/Y	-	A2->Y	F	AOI33XL	1	0.3	45	36	214	(-, -)
	drc_bufs12767/Y	-	A->Y	R	INVXL	4	1.2	20	20	234	(-, -)
	drc_bufs12766/Y	-	A->Y	F	INVXL	2	0.6	9	11	244	(-, -)
	g12505__9945/Y	-	A->Y	R	MXI2XL	2	0.6	22	18	262	(-, -)
	g12465__6783/Y	-	B->Y	F	NOR2XL	1	0.3	14	10	272	(-, -)
	g12461__6417/Y	-	B0->Y	R	AOI21XL	4	1.2	30	22	294	(-, -)
	g12435__5526/Y	-	A1->Y	F	OAI22XL	3	0.8	28	25	319	(-, -)
	g12424__1666/Y	-	B->Y	R	NAND2XL	2	0.6	14	14	333	(-, -)
	g12417__6260/Y	-	A1->Y	F	AOI211XL	2	0.3	19	18	351	(-, -)
	g12414__2398/Y	-	B->Y	R	NOR4XL	1	0.0	19	23	374	(-, -)
	alu_out_flag[0]	-	-	R	(port)	-	-	-	0	374	(-, -)

4. RESULT AND SIMULATIONS

```
C:\Users\datbt\AppData\Local\Temp\61a21e88-56a9-474e-9b20-c02e0f5eef8b_prj_alu-20260131T120326Z-3-001.zip.f8b\prj_alu>SYNTHESIS>_>SCRIPTS> alu_scripts.tcl  
1 =====  
2 Generated by: Genus(TM) Synthesis Solution 21.12-s068_1  
3 Generated on: Jan 31 2026 03:38:59 pm  
4 Module: ALU  
5 Operating conditions: PVT_1P32V_0C (balanced_tree)  
6 Wireload mode: segmented  
7 Area mode: timing library  
8 =====  
9  
10 Instance Module Cell Count Cell Area Net Area Combinational Buffer Inverter Flop Latch Clock-Gate Memory Macro Total Area Wireload  
11 -----  
12 ALU 338 517.788 0.000 479.484 3.420 34.884 0.000 0.000 0.000 0.000 0.000 0.000 517.788 ZeroWireload (S)  
13 | (S) = wireload was automatically selected  
14
```

4.RESULT AND SIMULATIONS

Instance: /ALU
Power Unit: W
PDB Frames: /stim#0/frame#0

Category	Leakage	Internal	Switching	Total	Row%
memory	0.00000e+00	0.00000e+00	0.00000e+00	0.00000e+00	0.00%
register	0.00000e+00	0.00000e+00	0.00000e+00	0.00000e+00	0.00%
latch	0.00000e+00	0.00000e+00	0.00000e+00	0.00000e+00	0.00%
logic	4.54363e-08	1.41547e-04	1.05999e-04	2.47592e-04	100.00%
bbox	0.00000e+00	0.00000e+00	0.00000e+00	0.00000e+00	0.00%
clock	0.00000e+00	0.00000e+00	0.00000e+00	0.00000e+00	0.00%
pad	0.00000e+00	0.00000e+00	0.00000e+00	0.00000e+00	0.00%
pm	0.00000e+00	0.00000e+00	0.00000e+00	0.00000e+00	0.00%
Subtotal	4.54363e-08	1.41547e-04	1.05999e-04	2.47592e-04	100.00%
Percentage	0.02%	57.17%	42.81%	100.00%	100.00%

Thanks for watching!!!